THE DEEPER LOVE OF GO

"A great way to dive into Go!"

—Max VelDink

EARLY ACCESS

Go 1.24

JOHN ARUNDEL

Contents

Praise for *The Deeper Love of Go*

A great way to dive into Go!
—Max VelDink

One of the best technical books I have read in a very long time.
—Paul Watts

John is both a superb engineer and, just as importantly, an excellent teacher / communicator.
—Luke Vidler

A fantastic example of good technical writing. Clear, concise and easily digestible.
—Michael Duffy

This book's writing style feels as if John is speaking to you in person and helping you along the way.
—@rockey5520

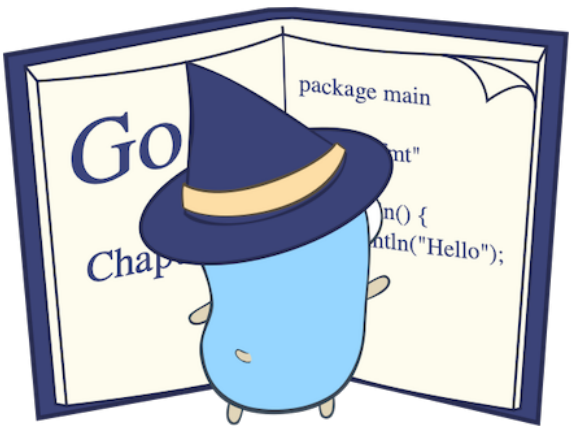
Very well written, friendly, and informative.
—Chris Doyle

John's writing is personable, human and funny—his examples are realistic and relevant. The test-driven instruction teaches Go in a deep, meaningful and engaging way.
—Kevin Cunningham

The book takes a very easy-to-follow, step-by-step approach to Go. The writing is very friendly and accessible. This would probably be my pick for the absolute beginner to computer programming who wants to learn Go.
—Jonathan Hall

A joy to read. Highly recommended for anyone starting with Go!
—Ryunosuke Zoran

Introduction



Hello, and welcome to learning Go! It's great to have you here.

What's this?

This book is an introduction to the Go programming language, suitable for complete beginners. If you don't know anything about Go yet, or programming, but would like to learn, you're in the right place.

If you do already have some experience with Go, or with programming in other languages, don't worry: this book is for you too! You'll learn some ideas, techniques, and ways of tackling problems that even many advanced Go programmers don't know. I hope to also help you fill in a few gaps in your knowledge that you may not even be aware of.

What you'll need

You'll need to install Go on your computer, if you don't have it already. Follow the instructions on the Go website to download and install Go:

- https://go.dev/learn/

While all you need to write and run Go programs is a terminal and a text editor, you'll find it very helpful to use an editor that has specific support for Go. For example, Visual Studio Code has excellent Go integration.

While not essential, it's a great idea to use some kind of *version control* software (for example, Git) to track and share your source code. I won't go into the details of how to install and use Git in this book, but if you're not already familiar with it, I recommend you learn at least the basics. Go to GitHub to find out more.

Where to find the code examples

There are dozens of challenges for you to solve throughout the book, each designed to help you test your understanding of the concepts you've just learned. If you run into trouble, or just want to check your code, each challenge is accompanied by a complete sample solution, with tests.

All these solutions are also available in a public GitHub repo here:

- https://github.com/bitfield/love

Each listing in the book is accompanied by a GitHub link with its name and number (for example, Listing hello/1, and clicking it will take you straight to the code for that program.

What you'll learn

By reading through this book and completing the exercises, you'll learn:

- How to manage data in Go using built-in types, user-defined struct types, and collections such as maps and slices
- How to write and test *functions*, including functions that return multiple results and error values
- How to use *objects* to model problems in Go, and how to add behaviour to objects using *methods*
- How to use *pointers* to write methods that modify objects, and how to use *types* and *validation* to make your Go packages easy to use

How to use this book

Throughout this book we'll be working together to develop a project in Go. Each chapter introduces a new feature or concept, and sets you some goals to achieve. Goals are marked with **GOAL** in bold.

Each goal also comes with one or more hints, in case you'd like a little assistance, and once you've either solved the problem or given up in disgust, there's a complete solution for you to look at. Don't you hate books that have questions, but no answers? So do I.

The love of Go

Go is an unusually fun and enjoyable language to write programs in, and I hope to communicate something of my own love of Go to you in this book. It's a relatively *simple* language, which isn't the same as easy, of course. Programming is not easy, and no language makes it easy. Some just make it a little more fun than others.

So let's get started!

1. Happy Fun Books

> *Do not taunt Happy Fun Ball.*
>
> —"Saturday Night Live", February 1991

Welcome aboard! It's your first day as a Go developer at **Happy Fun Books**, a publisher and distributor of books for people who like cheerful reading in gloomy times. I'm so glad you could join us, because we have a very important job for you: you'll be helping to build our new online bookstore using Go.

Hello, world

Don't worry if that sounds a bit intimidating: we'll start from scratch, and work our way up in easy stages. Actually, that's what experienced programmers do, too, if they're wise.

In other words, we don't start by trying to build something complicated like an e-commerce system. That's much too difficult. Instead, we start with the simplest thing we can imagine, and get *that* working first. Then we add a tiny increment to it that gets us a little bit closer to what we need to end up with.

Our first program

Let's start with about the simplest program imaginable: one that just prints a message. If we can write, run, and understand that program, we'll have covered a lot of useful ground already.

Here's the program:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world")
}
```

(Listing `hello/1`)

Traditionally, the message we print when we write our first program in any language is "Hello, world". Seems appropriate, doesn't it? And you can see this text in the program code. Don't worry about the other stuff for the moment: let's just run the program first, then analyse how it works.

Make a new folder on your computer—it doesn't matter where—and use your text editor to create a file named `main.go`. Copy the example program to this file and save it.

Now open your terminal application in this folder and run the following command:

```
go run main.go
```

If all goes well, you should see a friendly message:

```
Hello, world
```

Packages

Now let's take a closer look at the program that produces this message. How does it work?

It begins with this:

```
package main
```

All Go code, it turns out, belongs to some *package*. A package is just an arbitrary way of grouping related bits of code together, so that they can be used as a single unit. You can name your packages whatever you like, but there always has to be at least one package named `main` if you want to produce a runnable program.

For any file containing source code, we need to tell Go what package it belongs to, which is why every Go file must begin with a *package clause*, as this one does.

Imports

Next comes this line:

```
import "fmt"
```

This is called a *declaration*: it declares something to Go about this program. So, what are we declaring here?

As we've just seen, a Go program must have at least a `main` package, but if you want to use other packages, you're welcome. All you need to do is add an `import` declaration like this, naming the extra package you want to use. In this case, it's one called `fmt`. We'll see what it's for in a moment.

Functions

Here's another declaration. It's telling Go about a new *function* that we're defining:

```
func main() {
```

The `func` keyword, followed by a name (in this case `main`), declares a new function. So what's a function?

Declaring a function

Just like a package, a function is a way of organising your code into a little unit and giving it a name. A single program can contain many packages, and each of those packages can contain many functions.

And, just as the `main` package is special, because it's the package that imports all others, the `main` function is special too. It's where the execution of your program will actually start, so it's the function that calls all others, if there are any others.

Notice that this line ends with an opening curly brace, `{`. That means "Here comes the function body".

Go considers everything following the opening curly brace to be part of the *body* of the function we're defining, until it sees a corresponding closing brace, which ends the function definition.

Calling a function

Our `main` function body is nice and simple: it just contains this single line:

```
fmt.Println("Hello, world")
```

This is a *statement*. Whereas a declaration (such as the `import` declaration we just saw) just gives Go some descriptive information about our program ("We're going to use the `fmt` package"), a statement tells Go to *do* something: "Print this message!", for example.

To call a function, we write its name, prefixed by the name of the *package* it belongs to (`fmt.Println`), and followed by a list of *arguments* in parentheses ("Hello, world").

`fmt`, which Go programmers like to pronounce as "fumpt", deals with formatting text in various ways, including printing it to the terminal. And the specific function we're using here is `Println`, which is short for "print line". It prints whatever message we give it:

```
Hello, world
```

If we'd written a different message here, then that message would be printed instead when the program runs. Try changing the message, running the program again with `go run main.go`, and seeing what happens.

So we've already learned quite a bit from even this fairly short program: how to declare a package, how to import other packages, how to define a function, how to call a function, and how to pass arguments to it. We also learned one way to print things, which is handy.

A challenge

Now here's your first code challenge: to extend this program in an interesting way, using what you've just learned.

Creating a `hello` function

GOAL: Modify this program to add a declaration for a new function named `hello`. Your function should print the same "Hello, world" message that we've just seen. Modify the `main` function so that, instead of calling `fmt.Println` to print the message, it calls your new `hello` function. Check the result by running the `go run main.go` command again.

If you're not sure what to do, or you'd like a hint, read on.

HINT: Let's start with how to declare our new `hello` function. You know that the `func` keyword introduces a new function, and you've seen how it's used to declare `main`. You can use it in exactly the same way to declare your `hello` function. Can you see what to do?

Inside the curly braces marking the *body* of the `hello` function, we need some code that prints the "Hello, world" message. Well, you already know how to do that: the exact same way that `main` does it.

Finally, you need to modify the existing `main` function so that it calls `hello` instead of `fmt.Println`. And you know that you can call a function by writing its name, followed by parentheses (if there are no arguments, then you don't need to put anything between the parentheses, but they still need to be there).

If that function is in a different package, like `fmt`, then we need to use that package name as a prefix to the function call, but that's not the case here. When the function we're calling is in the "current" package—the one we're in right now—then we can just give its name, without a prefix.

SOLUTION: Well, here's how I did it. See how it compares to your version:

```
package main

import "fmt"

func hello() {
    fmt.Println("Hello, world")
}

func main() {
    hello()
}
```

(Listing `hello/2`)

Function ordering

By the way, it doesn't matter in what *order* you define your functions: you don't have to define the function before you can call it. You can write your functions in whatever order you think is most logical for people reading the program. To me, that usually means putting the most important or commonly-used functions first, but it's up to you.

Dealing with data

So, do we have a bookstore yet? Not quite, but we're on the way. What's the least we could do that would still be of some use to the staff of Happy Fun Books? Well, one thing we could do is, instead of printing "Hello, world", we could print a list of books that are in stock. How would we do that?

Simply printing a fixed message would be no good here, because Happy Fun Books has many branches, and the stock at each branch changes frequently. While the *logic* of our program ("print all the books") won't change, the specific *data* it deals with will vary from place to place and time to time.

Values

By "data", we really just mean some piece of information, like the title of a book, its price, or the address of a customer. These are all data *values*—bits of data.

So can we distinguish usefully between different *kinds* of data? Yes. For example, some data is in the form of *text* (a book title, for example). We call this *string* data, meaning a sequence of things, like "a string of pearls", only in this case it's a string of letters.

We've used a string value already in our "Hello, world" program. It was the message we passed to `fmt.Println`:

```
fmt.Println("Hello, world")
```

Notice that when this message is printed, it doesn't include the quote characters (`"`) at either end. These aren't part of the string itself: they're just markers, indicating to Go where the string begins and ends.

A piece of data like `"Hello, world"` is called a *value*. Suppose our bookstore is very small, and only has one book in stock at the moment. We could print its details by writing them as a string value:

```
package main

import "fmt"

func main() {
    fmt.Println("Books in stock:")
    fmt.Println("'Master and Commander', by Patrick O'Brian")
}
```

(Listing `books/1`)

If you want to run this program, you can create a new folder for it, open a new `main.go` file, and copy this code into it. Or you can edit the "Hello, world" program we wrote earlier in this chapter, so that it looks like this one. Then use `go run main.go` as you did before.

Notice what's different about this program. We have two statements in the `main` function now, not just one. They're both function calls, and they both call `fmt.Println`, but with different arguments—different *values*.

And we said that, while the logic of the program won't change, the data might. We probably won't need to change the "Books in stock:" message, but we certainly will need to change the second message, about the specific book we have in stock.

To make that easier, we'll store our book information in a *variable*. Let's see how to do that.

Variables

As the name suggests, variables are for storing data that might change (by contrast, values that don't change during the running of the program are called *constants*).

A variable is a way of giving a name to a data value, so that we can refer to it by that name later. Just as we declared a new function by using the `func` keyword, we can declare a new variable using the `var` keyword, like this:

```
var book = "'Master and Commander', by Patrick O'Brian"
```

You could think of a variable as being like a little box where we can keep bits of data—values—so they don't get lost. Each box has a label, so we know what's inside. In this case, the label is `book`, and whenever we use that name in the rest of the program, Go will open the box and retrieve the value for us.

For example, we could write:

```
fmt.Println(book)
```

Notice that this time, there are no quotes around the word `book`. That's because we're not asking Go to print the *string* "book". Instead, we're saying "Find the variable named `book`, and print the value stored inside it."

So here's a new version of the program that stores the book information in a variable:

```
package main

import "fmt"

```

```
func main() {
    fmt.Println("Books in stock:")
    var book = "'Master and Commander', by Patrick O'Brian"
    fmt.Println(book)
}
```

(Listing books/2)

On the face of it, this doesn't seem much more useful than what we did before, which was giving the book information as a quoted string. By the way, that kind of value is called a *literal*, as in "It's literally this string!"

The change we've made here is to first *assign* that literal value to a variable, and then use the *identifier* `book` to refer to that variable later on. The effect of the program is the same as when we just used a literal, but that's about to change!

Assigning values to variables

The point about variables, of course, is that we can *vary* them: we can assign a new value to them, which will replace the old one.

What do you think the following program will print?

```
package main

import "fmt"

func main() {
    fmt.Println("Books in stock:")
    var book = "'Master and Commander', by Patrick O'Brian"
    fmt.Println(book)
    book = "'A Morbid Taste for Bones', by Ellis Peters"
    fmt.Println(book)
}
```

(Listing books/3)

Let's run it with `go run main.go` and see if you're right:

```
Books in stock:
'Master and Commander', by Patrick O'Brian
'A Morbid Taste for Bones', by Ellis Peters
```

A well-stocked bookstore indeed! So, once we've created the new variable `book` with our

`var` declaration, we can then give it a new value any time we like by writing an *assignment* statement:

```
book = "'A Morbid Taste for Bones', by Ellis Peters"
```

Creating your own variables

So, here's another challenge for you, to help consolidate what you've learned.

GOAL: Write a Go program that creates two string variables, `title` and `author`, and gives them values: use the title and author of one of your own favourite books. Print these out. Then set `title` and `author` to different values, corresponding to another book. Print these out too.

HINT: We won't need any new function this time, though you can write one if you like. The important thing is to declare our variables and give them their initial values, and you know how to do that, using the `var` keyword. You also know how to print them, by calling `fmt.Println` and passing it the name of the variable you want to print.

Then we need to assign new values to both variables, and you've seen how to write an assignment statement to do that. All set?

SOLUTION: Here's my version:

```
package main

import "fmt"

func main() {
    fmt.Println("Books in stock:")
    var title = "The City & The City"
    var author = "China Miéville"
    fmt.Println(title, "by", author)
    title = "The Sugar Barons"
    author = "Matthew Parker"
```

```
    fmt.Println(title, "by", author)
}
```

(Listing books/4)

I cheated a little bit here, because I didn't tell you that you can pass *multiple* arguments to `fmt.Println` (or to any function), by separating them with commas. Well, now you know!

But if you printed `title` and `author` with consecutive calls to `fmt.Println`, that's totally fine. This way of doing it just saves a bit of typing, that's all.

Adding a `printBook` function

Notice that we repeated the exact same line of code here twice:

```
fmt.Println(title, "by", author)
```

You might be wondering: could we declare some *function* that does this instead, and then call it with different values, just like the way we call `fmt.Println` with different values?

Certainly we could:

```
package main

import "fmt"

func main() {
    fmt.Println("Books in stock:")
    var title = "The City & The City"
    var author = "China Miéville"
    printBook(title, author)
    title = "The Sugar Barons"
    author = "Matthew Parker"
    printBook(title, author)
}

func printBook(title, author string) {
    fmt.Println(title, "by", author)
}
```

(Listing books/5)

Declaring function parameters

Notice that our new function `printBook` is slightly different to the `hello` function we created earlier. Unlike that previous function, this one declares *parameters*—that is, it expects arguments to be passed when it is called.

```
func printBook(title, author string)
```

In this function declaration, we're telling Go that when we call this function, we will pass it two values, to which we'll assign the working names `title` and `author`. Also, both values contain the same kind of data, namely `string`.

The parameter names `title` and `author` happen to be the same as the ones we used for the corresponding variables in the `main` function, but this is just a coincidence. We could have written, for example:

```
func printBook(nameOfBook, personThatWroteIt string)
```

But it makes sense to use the same names for variables that contain the same data, doesn't it? And that's exactly what they are. Inside the `printBook` function, the names `title` and `author` are available for us to use as variables, and they'll contain whatever was passed to the function at the place it was called.

Here's what we know so far

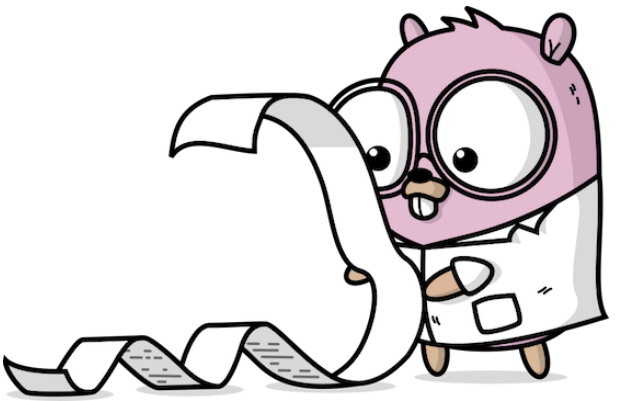
We've covered some really important ground in this chapter. You can now read any Go program and understand what package it belongs to, what other packages it imports, and what functions it declares.

You know that functions consist of statements, and you know about one kind of statement, which is a function call. You also know how to pass arguments to a function (like the message "Hello, world").

You've learned about how to deal with data, such as string values, how to store data in named boxes called "variables", how to declare function parameters, and how to use those parameters in the body of the function.

In the next chapter, let's extend this knowledge to cover more different *types* of data.

2. Data and types



In the previous chapter we did some interesting things with string data, but is that the only kind of data we can use in a program? Certainly not. There are all sorts of other kinds of data we can work with. For example, numbers.

New data types

A well-stocked bookstore will certainly contain more than one copy of some books, so perhaps we'd like to keep track of that information, too. As with the title and author, we can store this in a variable. Let's call it `copies`:

```
var title = "Killers of the Flower Moon"
var author = "David Grann"
var copies = 15
```

Numeric data

Notice that there are no quotes around the value `15`. That's because this isn't a piece of text: it's not the string `"15"`, it's the *number* 15.

Go considers these as two distinct kinds of information: two distinct *data types*. The value `"David Grann"` is a string literal; `15` is called an *integer* literal (more about integers in a moment).

Now we can print this stock level information out as part of the book details:

```
fmt.Println(title, "by", author, "-", copies, "copies")
```

Here's the result:

```
Killers of the Flower Moon by David Grann - 15 copies
```

Great! We should be able to cope with any sudden surge in demand for this excellent book.

Parameter types

Of course, now that we know how useful abstractions are, we'd like to add this new piece of data to our `printBook` function. Here's the current declaration of `printBook`, showing the parameters it can take:

```
func printBook(title, author string)
```

We can add `copies` to this list of parameters, but we also have to tell Go what data type it is. It's not a string, so we can't say `string`. What should we put here?

The `int` type

In fact, Go has more than one built-in data type for dealing with numbers. The most commonly used one is called `int`, short for "integer". The integers (Latin for "whole") are, as you probably know, the whole numbers.

So let's declare `copies` as a parameter of type `int`:

```
func printBook(title, author string, copies int) {
    fmt.Println(title, "by", author, "-", copies, "copies")
}
```

(Listing `books/6`)

Now all we have to do is remember to add the `copies` information each time we call `printBook` in our `main` function:

```
func main() {
    fmt.Println("Books in stock:")
    var title = "The City & The City"
    var author = "China Miéville"
    var copies = 1
```

```
    printBook(title, author, copies)
    title = "The Sugar Barons"
    author = "Matthew Parker"
    copies = 2
    printBook(title, author, copies)
}
```

(Listing books/6)

We've imagined a variable (or, equivalently, a function parameter) as being like a little box that you can put different data values into, and the box has a name on it (for example, `copies`). Now we know that the box also has a label on it identifying what *kind* of data it can hold: for example, `string` or `int`.

Variable declarations

So far, each time we created a new variable (such as `title`) we've declared its name and assigned its value in a single statement, like this:

```
var title = "The City & The City"
```

That's convenient, but we can also declare the variable *without* explicitly giving it a value:

```
var title string
```

Notice that we have to tell Go what type the new variable is here (`string`). We didn't have to say that when we assigned a value, because Go is smart enough to work out what the type *must* be, based on that value.

Here, there's no such clue, so we have to state the type explicitly: `string`. Or, if we wanted to declare an integer variable, we'd say `int`:

```
var copies int
```

Later, we can assign a value to our new variable with an assignment statement of the kind you've already seen:

```
title = "The City & The City"
copies = 1
```

Zero values and default values

You might have wondered what happens if we declare a new variable using the `var` keyword, but then try to refer to it before we've actually assigned any value to it. What happens when we try to print the variable, for example?

Let's see:

```
var x int
fmt.Println(x)
// Output:
// 0
```

How interesting! It turns out the variable has a *default value*, which is what it contains before we explicitly put anything in it. What that value actually *is* depends on the type.

Each type in Go has a *zero value*, which is the default value for variables of that type. To put it another way, when you order a new box, the box doesn't arrive empty. You get a free gift inside it: a copy of the zero value for that type of box.

For `int`, the zero value is the number `0`, which makes sense, but what about other types?

The zero value for `string` is the empty string, `""`, which also seems reasonable. Check this for yourself:

```
var s string
fmt.Println(s)
// Output:
//
```

No output, but that makes sense: the string `s` is empty, so there's nothing to print.

Type checking

What's the point of having types, then? Well, I'm glad you asked.

In theory, we could just store all information as strings, but then we could easily get mixed up between one variable and another. The type system helps catch mistakes like this, because Go provides *type checking* for us: it won't let us assign values of one type to variables of another type, for example.

Type mismatch errors

What happens if we try to put a value of one type into a box that's defined to hold a different type? Presumably that won't work, but let's see what happens.

At the risk of making Go angry, we'll try to put an `int` value into a `string` variable, and vice versa:

```
title = 42
copies = "None"
```

As expected, we get some errors on trying to run this program:

```
cannot use 42 (untyped int constant) as string value in assignment
cannot use "None" (untyped string constant) as int value in assignment
```

Ignoring the stuff about "untyped constants" for now, this means, effectively:

```
cannot put an 'int' value into a box marked 'string'
cannot put a 'string' value into a box marked 'int'
```

And that makes sense. Even if we *could* do this kind of thing, it would be a bad idea. With data, there's a place for everything, and everything should be in its place. If you put salt in the pepper grinder and pepper in the salt shaker, it will just confuse and annoy your dinner guests, and they probably won't come again.

A plausible bug

Type checking is useful for when we make genuine mistakes, not just when we're playing practical jokes on Go like this. For example, we might forget the correct order of arguments to the `printBook` function, and mix them up, something like this:

```
printBook(title, copies, author)
```

This is the sort of *bug* (mistake) that happens all the time in programming, so it's handy that Go can spot it for us and report the problem:

```
cannot use copies (variable of type int) as string value...
```

We're not really *trying* to use an `int` as a string value, of course, but Go doesn't know that.

It can't say "Hey, you dummy, you got your arguments in the wrong order," even though that's the true source of the problem. But it can at least report that there's *something* screwy about this code, and it's up to us to then look closer and figure out what we did wrong.

Structured data

We're making progress towards our bookstore application. We've now figured out how to store various bits of data about a book in variables, using the appropriate type for each one: `string` for the title and author, `int` for the number of copies in stock.

So far, so good. But we know we'll probably need to store several more pieces of data about each book in the finished program. For example, we might want to know the book's ISBN number, publisher, year of publication, and so on. We'd certainly like to be able to store its price, and perhaps a brief description of its contents, even a cover image.

That's a lot of variables for one book! It's already a little inconvenient to use a function such as `printBook`, because we have to pass each variable as a separate argument: `title`, `copies`, `author`, and perhaps more in the future.

Introducing structs

Ideally, since all these variables relate to a single book, we'd like to be able to group them together into *one* variable. Not just that, but we'd like to maintain the *structure* of that variable: the fact that there's a string containing the book's title, another string containing its author, and an integer number of copies. Then we could retrieve any of those bits of information individually when we want them, while still having the convenience of passing a single value to a function that deals with books.

Fortunately, Go has a kind of data type that works exactly this way. It's called a *struct* (as in "structured data"). Let's see how to use it.

Declaring a new struct type

We want to create a struct type that stores data about a book, and we can do that with a *type declaration* like this:

```
type Book struct {
    Title  string
    Author string
    Copies int
}
```

The `type` keyword introduces a new type, and the name `Book` is what we're going to call it. Then, the keyword `struct` tells Go that this is going to be a structured type, which means the next thing we need to do is list the individual *fields*—the variables that make up our book information.

The opening curly brace `{` prefaces the list of fields, just as it marks the start of the function body when we're declaring a new function. Generally in Go, an opening curly

brace means “Here comes some stuff”, and the corresponding closing brace means “That’s the end of that stuff”.

Within the block of code marked by the opening and closing braces, we list each field, giving its name (for example `Title`) and the type of data it stores (for example `string`). They can all be different types, and there can be as many fields as we want, but we just need these three for now.

Great, so we created a new struct type named `Book`, but what now? How do we actually use it?

Struct variables and values

Well, the first thing we should do is declare a new variable of this type:

```
var book Book
```

Just as we previously declared new variables of type `string` or `int`, which are built-in types, we can use our newly-defined `Book` type to create a variable in the same way. Now let’s assign it a value:

```
book = Book{
    Title: "Sea Room",
    Author: "Adam Nicolson",
    Copies: 2,
}
```

Combining declaration and assignment

Again, this assignment statement works exactly like the previous ones we’ve written: there’s the name of a variable (`book`) on the left-hand side of the = sign, and a literal value on the right-hand side. In fact, we could also have written this in a single step, declaring *and* assigning in the same statement:

```
var book = Book{
    Title:  "Sea Room",
    Author: "Adam Nicolson",
    Copies: 2,
}
```

Notice that we now don’t have to mention the *type* of variable that `book` should be. Go is smart enough to infer it, because the only type of variable you could assign a `Book` value to would be a `Book` variable. Why repeat yourself if you don’t have to?

It's so common to combine variable declaration and assignment in Go programs that we have an even more concise way to write the same thing:

```
book := Book{
    Title:  "Sea Room",
    Author: "Adam Nicolson",
    Copies: 2,
}
```

Notice that the `var` keyword disappeared altogether, and instead of the `=` sign ("is assigned the value"), we have the strange-looking `:=` sequence ("is implicitly declared and assigned the value"). This "colon-equals" syntax is technically known as the *short declaration form*, but I've also heard it called the "walrus operator" (try looking at it sideways).

The short declaration form is common in Go code, so we'll use it instead of `var` from now on.

Struct literals

We know what string and integer literals look like, because we've used them before, but we have something new here: a *struct literal*. Specifically, a `Book` literal.

We write that, as you can see, by first of all giving the name of the type (`Book`), then using an opening curly brace: here comes the stuff. Namely, a list of *elements*: that is, individual bits of data.

In a struct literal, each element is a key-value pair: the name of some field (for example `Title`), followed by a colon, followed by the value we want to assign to that field (for example `"Sea Room"`), and finally a trailing comma.

Perhaps it doesn't really seem like all this gains us that much. I mean, we started out with three separate variables, and now we have one variable with three separate fields. What's the difference?

The main reason for grouping data values together into a struct is that it makes them easier to pass around (for example, to a function such as `printBook`). We get the convenience of having the book information in a single variable, but we also keep the flexibility of being able to access any of the individual fields by name when we want to:

```
fmt.Println(book.Title)
// Output:
// Sea Room
```

Accessing struct fields

Notice that we use the name `book.Title` to refer to this field (pronounced "`book` dot `Title`" when we're reading out loud).

It's not enough just to say `Title`, because of course there could be lots of *different* `Book` variables: which one do we mean? We have to identify the specific book first of all (`book`), then the field name prefixed by a period (`.Title`). Together, the two things identify a specific field on a specific `Book` variable.

Similarly, we can assign a new value to a struct field if we want to:

```
book.Copies = 1
```

And because a struct is essentially just a bunch of variables glued together, we can use those struct fields anywhere that we could use an individual variable:

```
printBook(book.Title, book.Author, book.Copies)
```

That's nice, but we can do better! Now that we can treat these pieces of book data as a single value, we could change our `printBook` to take just one parameter, instead of three. Can you work out how to do that?

Struct parameters

GOAL: Modify the `printBook` function so that it takes a single parameter representing all the information about a specific book. Try out the new function and make sure it prints the same output as before.

HINT: Well, you know how to declare a *variable* of type `Book`, so what about a function parameter? That works pretty much the same way, it turns out. See if you can guess what the syntax needs to be.

Once you have that parameter, you can refer to its individual fields using the dot notation we saw in the previous example: first comes the name of the parameter, then a period, then the name of the field we want.

SOLUTION: Here's one way we could write `printBook`:

```
func printBook(book Book) {
    fmt.Println(book.Title, "by", book.Author, "-", book.Copies, "copies")
}
```

(Listing books/7)

Let's try it out:

```
fmt.Println("Books in stock:")
book := Book{
    Title:  "Sea Room",
    Author: "Adam Nicolson",
    Copies: 2,
}
printBook(book)
// Output:
// Sea Room by Adam Nicolson - 2 copies
```

(Listing books/7)

Pretty neat! This not only makes it easier and less verbose to call `printBook`, and simplifies the function declaration, it's also a kind of future-proofing. As we add more fields to the `Book` struct in the future, we won't need to modify every place in the program where we call `printBook`; we can just change the function itself.

Even when we change the details of the `Book` struct in some way, such as adding a field, our calling code can stay exactly the same:

```
printBook(book)
```

The code of `printBook` will probably need to change, naturally, but that only occurs in one place, so it's a lot easier to find and update.

In fact, let's make a change now, just for fun. The `fmt.Println` function, as you probably noticed, prints all its arguments, separated by spaces. This is fine, but it's not always easy to see exactly what the result will look like, especially when there are lots of arguments.

It would be more convenient if we could specify a *format string*: a kind of template string containing the literal text we want ("by", "copies"), but also placeholders showing where the data value would go.

For example, suppose we used `%v` as a placeholder for values, we could write a format string like this:

```
"%v by %v - %v copies"
```

And now we also need to supply the data, of course, and since there are three `%v` placeholders, we need three pieces of data to fill them: `book.Title`, `book.Author`, and `book.Copies`.

So, `fmt.Println` has a companion function that works exactly like this: `fmt.Printf`. It takes a format string, possibly containing placeholders, and then as many pieces of data as needed to fill the placeholders (if any). Our example would look like this:

```
fmt.Printf("%v by %v - %v copies\n", book.Title, book.Author, book.Copies)
```

Note that `Printf`, unlike `Println`, doesn't automatically add a newline character to the end of our string. We can supply one, though, by including the sequence `\n` (for "newline") in the format string.

Formatted printing like this is extremely useful. You can imagine that, since there could be a lot of data, and the format string could be very long, this is a more flexible and friendly way of printing than `fmt.Println` provides.

Here's what we know so far

Now you know about some new types, including `int`, and *struct* types that combine different kinds of data into a single record. You've learned how to declare and assign variables in one statement, using the short declaration form, and you also know how to write functions that take structs as parameters, and access the data in individual fields.

Great job! In the next chapter, we'll continue our data-mangling activities, and in particular we'll see how to *test* functions that process data in various ways.

3. Tests

Creating abstractions, such as the `printBook` function, or the `Book` struct type, is a useful way to organise our programs and make them easier to understand. We don't have to write lots of duplicated code, and if we want to change the way something works, we can do it in just one place.

This makes maintenance easier, and you'll find that as a software engineer you spend far more of your time updating and fixing existing code than you do on writing brand new code. And we've proved this already: we've made several changes to the `printBook` function, for example, as our requirements evolved.

Self-testing code

The first time we write a piece of code, we can usually tell whether or not we've got it right: just run the program and make sure it works. But as soon as we start making changes, this *testing* process gets harder.

What does "it works" mean?

For one thing, it's not always clear what "it works" means. Even if the program doesn't crash, or report an error, or behave weirdly, it might be incorrect all the same. For example, we might have forgotten to print out the `Copies` field as part of the book information. That's a bug, but it would still look as though the program "works"—it just doesn't print everything it's supposed to.

Or suppose we make some minor change to the `printBook` function that, unexpectedly and by mistake, causes it to stop printing out the `Copies` information. We might run the program once or twice and think everything is fine. We weren't expecting the copies information to disappear, so we weren't specifically looking for it.

So, while *manual testing*—running the program and looking for anything weird—has some value, we'd like to have a more rigorous and less labour-intensive way of checking for bugs.

Really, the ideal thing would be for the program to be able to test *itself*. How would that work?

How could we test this?

Well, suppose we wanted to test the `printBook` function manually. What would we do? We might write a short program that creates a `Book` value and passes it to `printBook`. Then we could run the program and check that it produces the expected output.

With one or two small changes, we can automate this process to create what we want: *self-testing code*. For example, suppose we wrote a function `testPrintBook` that calls `printBook` and checks its result. If we had such a function, we could call it from the program's `main` function before doing anything else, and if it detected a problem, it could terminate the program.

So far, so good: let's think now about what the test function would actually do. How does `printBook` behave when it's working correctly? Well, it prints the given `Book` information to the terminal, in a certain specified format.

That's awkward to test, because printing is a one-way process: we can call `fmt.Printf` or `fmt.Println` to print something, but there's no corresponding function we can call to ask *what* was printed.

Designing for testability

Instead, let's re-jig the behaviour of `printBook` a bit. What if we changed `printBook` so that, instead of *printing* the `Book` data, it just returned a suitably-formatted string? Then we could always print that if we wanted to. In the test, we won't need to: we'll just compare it with the string we expect.

So, let's rename `printBook` to something like `BookToString`, and have it return a string instead of printing anything. We won't make any code changes yet; let's finish writing the test function first.

The tools we need

The plan is coming together, but we still need a couple more pieces. We need a way to *compare* two values, and we also need a way to exit the program if the comparison fails.

Conditional statements

Actually, when you think about it, the word "if" is doing a lot of work in that sentence, because it implies that for the first time our program could take one of two possible paths, depending on some condition. That may seem straightforward, but it represents quite a leap in our ability to write useful programs, and this important idea has a name: the *conditional statement*.

A conditional statement, as the name suggests, lets us make the execution of some part of the code conditional on the value of some expression. *If* the expression is true, the code is executed, and you may be pleased to learn that the keyword Go uses for this is simply `if`:

```
if x == 1 {
    fmt.Println("Looks like x is 1!")
}
```

Notice that, again, curly braces are used to indicate "here comes some stuff": in this case, the block of code to be executed if the condition is true.

Comparison operators

There's another new piece of syntax here, too: the *comparison operator* ==. Not to be confused with the single = sign that assigns a value to a variable, the == operator means "is equal to".

So we can now conditionally execute some code if two values are equal, but that's not quite what we need for the test. We want to do something (report a test failure, for example) if the values are *not* equal, which is the opposite.

Here's what that looks like:

```
if x != 1 {
    fmt.Println("Looks like x is NOT equal to 1!")
}
```

The `!=` operator, pronounced "not equals", or "not equal to", is just what we'll want to use to fail the test if `BookToString` returns something unexpected.

The `panic` function

We now have nearly all the pieces we need to construct a useful self-test function for `BookToString`. Here's the plan: we'll create a `Book` variable containing details about some book, we'll pass it to `BookToString`, and then we'll compare the result to some expected value using `if` and ==.

If the result is different to what we expect, we'll stop the program with an error message to indicate that something's gone wrong. To do this, we can use the built-in function `panic`:

```
panic("Looks like x is NOT equal to 1!")
```

The quaintly-named `panic` does just what we need here: it prints the given message to the terminal and immediately exits the program. So, if the self-test fails, we won't proceed to the normal operation of the program, because we'll know that the code is faulty and shouldn't be used until it's fixed.

Writing the test

Let's write the test function, then:

```
func TestBookToString_FormatsBookInfoAsString() {
    input := Book{
        Title:  "Sea Room",
        Author: "Adam Nicolson",
        Copies: 2,
    }
    want := "Sea Room by Adam Nicolson - 2 copies"
    got := BookToString(input)
    if want != got {
        panic("BookToString: unexpected result")
    }
}
```

(Listing books/8)

This is the most complicated function we've written so far, but if you read through it carefully, line by line, what it's doing is straightforward. It's exactly what we planned.

We create a `Book` variable (here named `input`)—remember the short declaration form using `:=`? Of course you do. Then we pass the value of that variable to the `BookToString` function, and assign the *result* of that function to a variable named `got`.

We then *compare* `got` with another variable we prepared earlier, `want`, which is just a string that represents what `booktoString` should return if it's working correctly. Using an `if` statement and the `!=` comparison operator, we call the `panic` function if `want` and `got` contain different strings.

There's one other thing about this test function that's worth looking at: its name!

`TestBookToString_FormatsBookInfoAsString`

What does this mean? It's telling us what the test is testing: that `BookToString` formats book info as a string. That might sound obvious, given the name of the function. Fine. Apparently we've picked a good name for the function: we *want* its purpose to be clear from the name.

And it's the same for this test function. We could have named it just `TestBookToString`, or something like that, and it is at least clear *which* function is being tested. But it's not clear what we're testing *about* it.

The extra words in the name make that explicit: we're testing that it does, indeed, format the given book info as a string (and the test code also shows exactly what that format is supposed to be).

Testing the test

Very promising! There's just one problem: we haven't actually written the `BookToString` function yet. Let's tackle that next.

In fact, let's again break down the problem into smaller stages. Logically, if `BookToString` contains a bug, we want the *test* function to panic and stop the program. But how do we know that will actually happen? After all, there might be a bug in the test function too!

To validate the test function, we should first write a version of `BookToString` that we *know* is incorrect, and make sure that the test function detects that fact. If it doesn't, we have a problem with the test.

On the other hand, assuming the test *does* detect the bug, then we can have some confidence in the correctness of our test, and we can go ahead and write the real version of `BookToString` which (we hope) will actually work.

Functions that return results

So we'll start with the buggy version first. We already know that we need to change the function's name, and declare that it now returns a `string` result instead of nothing. So here's another new piece of syntax:

```
func BookToString(book Book) string {
```

Recall that when we declare a function using the `func` keyword, we have to list all the parameters it takes inside a pair of parentheses after the function name. If the function returns any results, we have to list them after the closing parenthesis, but before the opening curly brace marking the start of the function body.

Here, we're saying that the `BookToString` function returns one result, of type `string`—which makes sense. Notice that, while we have to specify the *type* of the result, we don't need to give it a name.

The `return` statement

Up to now, none of the functions we've written actually returned any results. Now we need to do that, and merely declaring in its signature that the function *will* return a result isn't quite enough. To actually return something, we have to write a `return` statement:

```
func BookToString(book Book) string {
    return ""
}
```

A `return` statement causes the function to stop executing at that line, and pass the specified value back to its caller. Often this is the last line in the function, but it needn't be—a function can have more than one return statement and they can occur anywhere in the function body.

So, what are we returning in this case? Just the empty string, `""`. That's obviously incorrect, because it ignores the book information supplied in `book`, and always returns the empty string no matter what. But that's what we wanted: a version of `BookToString` that we know in advance won't work.

Silly, perhaps, but you get the point: if the test function doesn't detect this bug, what bugs *would* it detect? So we'll start with this one.

Running the test

To enable our new test machinery, let's replace our existing `main` function with one that (for the moment) simply calls the test function:

```
func main() {
    TestBookToString_FormatsBookInfoAsString()
    fmt.Println("It's all good!")
}
```

When we run this program, it will print one of two possible messages. If the test fails (that is, if the test succeeds in detecting our deliberate bug), then it will print the panic message. Otherwise, it will print "It's all good!"

Let's see what happens:

```
go run main.go
```

```
panic: BookToString: unexpected result
```

There follows more output, which is just to help with diagnosing the `panic` problem, but we don't need that in this case, because we know exactly what's wrong. `BookToString` is returning the wrong answer, and our test detected that fact. Excellent!

Let's go ahead and write a better version of `BookToString`, then, and we'll use the test to check whether or not we got it right. Here's the old function we had before:

```
func printBook(book Book) {
    fmt.Printf("%v by %v - %v copies",
        book.Title, book.Author, book.Copies)
}
```

Previously, we relied on `fmt.Printf` to do both the string formatting *and* the printing. We still need the formatting behaviour, but now we just want to get the result as a string, without printing it.

The `fmt.Sprintf` function

There's another function in the `fmt` package to do exactly that: `fmt.Sprintf`. Just like `Printf`, it takes a format string with placeholders, plus the data, but it doesn't print anything. It just creates a string with the format we asked for.

So we could write something like this:

```
func BookToString(book Book) string {
    s := fmt.Sprintf("%v by %v - %v copies",
        book.Title, book.Author, book.Copies)
    return s
}
```

But, since we don't actually do anything with the variable `s` other than immediately returning it, we can drop the variable entirely and write just:

```
func BookToString(book Book) string {
    return fmt.Sprintf("%v by %v - %v copies",
        book.Title, book.Author, book.Copies)
}
```

(Listing `books/8`)

Passing the test

Let's run the program again now and see if the self-test passes:

```
go run main.go
```

```
It's all good!
```

Excellent news. And we can imagine writing other tests for other functions that might be in our program. It's very reassuring to know that if we ever accidentally introduce a mistake into the code somewhere, the self-test system will catch it and let us know.

The `testing` package

It's so common and useful to write self-test functions like this that Go actually provides some built-in facilities to help us. Let's see how to use them.

Declaring a test

First, to keep things tidy, we'll move our test function to a separate source file, `main_test.go`, and make a few small tweaks:

```
package main

import "testing"

func TestBookToString_FormatsBookInfoAsString(t *testing.T) {
    input := Book{
        Title:  "Sea Room",
        Author: "Adam Nicolson",
        Copies: 2,
    }
    want := "Sea Room by Adam Nicolson - 2 copies"
    got := BookToString(input)
    if want != got {
        t.Fatal("BookToString: unexpected result")
    }
}
```

Here's what changed:

- We imported the `testing` package, which is part of the standard library
- The test function now takes a parameter, which we name `t`. We'll see what this is for in a moment.
- Instead of `panic`, to report the test failure we call `t.Fatal` (so that's what the `t` was for!)

Otherwise, everything is the same. There's one more step we need to take before we can run this test: creating a *Go module*.

Creating a module

You already know what a package is, and a module is simply a group of packages that live together. In fact, there needn't be more than one package, and in our case there isn't. However, every package needs to be part of a module, so let's call this one `books`. Run the following command:

```
go mod init books
```

```
go: creating new go.mod: module books
go: to add module requirements and sums:
        go mod tidy
```

Notice that a new file has been created, named `go.mod`, with the following contents:

```
module books

go 1.22.0 // or whatever Go version you're using
```

(Listing books/9)

It's this file that identifies the module: it tells Go, "All the packages in this folder or its subfolders are part of module `books`".

The `go test` command

Great. Here comes the payoff. We can now run *just* the test, without having to run the whole program:

```
go test
```

```
PASS
ok      books       0.204s
```

The output "`PASS`" tells us that the test passed, which is reassuring. Just for fun, though, let's see what gets printed when a test fails.

Detecting a bug

To do that, let's deliberately insert a bug into the `BookToString` function (which should still be in your `main.go` file). Change it as follows:

```
func BookToString(book Book) string {
    return fmt.Sprintf("OOPS %v by %v - %v copies",
        book.Title, book.Author, book.Copies)
}
```

We just inserted the word "OOPS" at the beginning of the format string, so that `BookToString` will always return a result beginning with "OOPS", whatever the input.

That *should* make the test fail, if anything does, so let's see:

`go test`

```
--- FAIL: TestBookToString_FormatsBookInfoAsString (0.00s)
    main_test.go:14: BookToString: unexpected result
FAIL
exit status 1
FAIL    books       0.248s
```

Interpreting the failure output

Interesting! We saw a lot more output this time, which makes sense: there's something to report.

First, we see `FAIL` followed by the name of the test function that failed. When we have lots of test functions (and we will), it'll be useful to know which one failed. Indeed, we also see the name of the source file where the test is defined (`main_test.go`) and the exact line where the failure happened: line 14.

That's where the call to `t.Fatal` is, which helps us zero in even more closely on the problem. We know that this line is never executed unless `want != got`. So, `BookToString`'s result was different to what we expected. But different how?

It's not easy to see that from the failure message, so let's change the message to report a few more details:

```
if want != got {
    t.Fatalf("want %q, got %q", want, got)
}
```

Notice that now we use `t.Fatalf` instead of `t.Fatal`, and that just like `Printf` and `Sprintf`, it takes a format string plus some data.

In that format string, we've used a new kind of placeholder. Instead of `%v`, which prints any value, we've used `%q`, which is specifically for strings, and it prints the string inside double quotes.

A better failure message

Let's run the test again and see what it produces:

```
--- FAIL: TestBookToString_FormatsBookInfoAsString (0.00s)
    main_test.go:14: want "Sea Room by Adam Nicolson - 2 copies",
    got "OOPS Sea Room by Adam Nicolson - 2 copies"
```

Now we can see what was supposed to happen, along with what actually *did* happen. This makes the bug very clear. Of course, in this case we knew what it was all along, but other bugs might creep in later, so this reporting will be very helpful in future.

The naming of tests

Something else that's helpful to see is that extra-informative test name we carefully crafted earlier:

`TestBookToString_FormatsBookInfoAsString`

This is reminding us what the test *was*, by expressing the behaviour we want as a sentence: "`BookToString` formats book info as string".

Since the test is failing, we know that's not happening, or at least it's not happening correctly—in this case, because the format is wrong. Now you can remove the bug you deliberately inserted (so take the "OOPS" out of `BookToString`) and make sure the test passes again.

Tests as docs, with `gotestdox`

I find this way of naming tests very useful, not just when the test fails, but also when it passes. A passing test means that the "behaviour sentence" is now a true and fair description of what the function does. It's a kind of documentation, almost.

When you have several tests, which we eventually will, their names will form an increasingly comprehensive description of the package. At least, they will if we choose those names carefully.

Viewing "tests as docs" is such a neat idea that I wrote a little tool to help me do it, called `gotestdox`. Let's install it now:

```
go install github.com/bitfield/gotestdox@latest
```

If we run it in our project directory, it will run all our tests (all one of them, so far), and print out their names *as* sentences, adding spaces as necessary:

```
gotestdox
```

```
books:
 ✔ BookToString formats book info as string (0.00s)
```

Very nice! Running `gotestdox` over my tests every so often reminds me to give them names that make sense when read as sentences.

Updating `main`

Now we've taken advantage of Go's built-in test facilities, we don't need our old home-grown self-test. We can update `main` so that it does something useful now: printing the details of a book!

```
func main() {
    book := Book{
        Title:  "Engineering in Plain Sight",
        Author: "Grady Hillhouse",
        Copies: 2,
    }
    fmt.Println(BookToString(book))
}
```

(Listing books/9)

Let's try it out:

```
go run main.go
```

```
Engineering in Plain Sight by Grady Hillhouse - 2 copies
```

Great! Now we're not wasting users' time by running self-tests every time they want to use the program. As the developer, we can run the tests any time we want, but the program we ship to users won't include them.

Making changes, guided by tests

Time for another challenge, to give you a little practice in *using* tests to help you modify the program. We're going to make a change to the way that `BookToString` formats the book information, but we're going to do that in two separate steps.

Updating the test

First, we'll change the *test* function to expect a different string format: the one we want. After all, in a sense, this test is the official "specification" of what `BookToString` is supposed to do. If we want to change the behaviour, we should change the specification too.

Once we've changed the test, we should run it and make sure we *see it fail*. Why? Well, suppose that we modified the test to expect some different result than `BookToString` produces now. How could it possibly pass, until we've made the corresponding change to `BookToString`?

It couldn't. So that's the point: in order to verify that the test itself is working, we should check it against a "known bad" version of `BookToString`, and make sure that it fails in the way we expect.

Only then will we go ahead and change `BookToString` so that the test passes once more. That way, we know that even though we've changed the program, both the test *and* the function are still correct.

Over to you for the first step:

GOAL: Modify the test function so that it expects `BookToString` to produce a result like this:

```
Sea Room by Adam Nicolson (copies: 2)
```

Failing the test

Don't change anything else yet, just the `want` value in the test function. Run the test and make sure that you get the following output before moving on:

```
--- FAIL: TestBookToString_FormatsBookInfoAsString (0.00s)
    main_test.go:14: want "Sea Room by Adam Nicolson (copies: 2)",
    got "Sea Room by Adam Nicolson - 2 copies"
```

SOLUTION: Here's what your test function should look like after the change:

```
func TestBookToString_FormatsBookInfoAsString(t *testing.T) {
    input := Book{
        Title:  "Sea Room",
        Author: "Adam Nicolson",
        Copies: 2,
    }
    want := "Sea Room by Adam Nicolson (copies: 2)"
    got := BookToString(input)
    if want != got {
        t.Fatalf("want %q, got %q", want, got)
    }
}
```

Updating the function

The test's job is to detect bugs in the `BookToString` function, and it's currently detecting one! Namely, that the string returned by `BookToString` is in a slightly different format from the one we require. Let's go ahead and fix that problem now.

GOAL: Modify `BookToString` so that it passes the test!

HINT: Okay, let's look at the version of `BookToString` we have right now:

```
func BookToString(book Book) string {
    return fmt.Sprintf("%v by %v - %v copies",
        book.Title, book.Author, book.Copies)
}
```

Changing the format

We know this needs to change, but how? Well, the part that dictates the format of the returned string is the first argument to `fmt.Sprintf`:

```
%v by %v - %v copies
```

If you think about it, the *data* we're putting into this string is exactly the same—it's just that the placeholder indicating where the "copies" information will go needs to move, and we need to tweak the surrounding text a little. See what you can do. If the test is failing, use the information it prints to find what's wrong.

SOLUTION: Here's the new format string we need:

```
%v by %v - (copies: %v)
```

Pretty straightforward, isn't it? We can think of the format string as a "picture" of how the final string will look, but just with `%v` standing in for the actual data.

Passing the test

The order in which the `%v` placeholders occur in the format string is the same order that we need to pass the data values as the other arguments to `fmt.Sprintf`, and that hasn't changed. So, the working version of `BookToString` should look something like this:

```
func BookToString(book Book) string {
    return fmt.Sprintf("%v by %v (copies: %v)",
        book.Title, book.Author, book.Copies)
}
```

(Listing books/9)

And let's check that the test actually passes:

```
go test
```

```
PASS
```

```
ok      printbook       0.199s
```

Finally, let's see what the real program produces:

```
go run main.go
```

```
Engineering in Plain Sight by Grady Hillhouse (copies: 2)
```

Creating a package

You learned in the first chapter that a Go program can import and use functions from other packages, such as `fmt` or `testing`. And, in turn, Go programs can *provide* packages that can be imported and used in other programs.

`main` is not importable

So far, all the functions we've written, apart from tests, have been in the `main` package, and you'll recall that `main` is special because it's the only package that can be compiled into an *executable binary*, that is, a runnable program.

But there's something else special about `main`, which is that it can't be imported by any other package. That makes sense if you think about it a little: in order for someone to produce a runnable program, they'd have to have their *own* `main` package. If they tried to import ours too, there would be two packages with the same name in the same program, and Go doesn't allow that.

So, as useful as what we've already built for Happy Fun Books is, it's only useful in this one specific `main` package that we've written. We couldn't write a *different* program that imported our `main` package and used its `BookToString` function, for example, and nor could anyone else.

The `books` package

That would be a shame, so let's fix it. We'll move the "general bookstore functionality" code we've written into an importable package (let's call it `books`, since that's the name of our module too). Then, we'll import that package *ourselves* in `main` and use it.

First, we'll create a new file in the project folder named `books.go`, and add a line declaring that it's part of a package called `books`:

```
package books
```

Then, we'll move over the `Book` struct and the `BookToString` function from `main.go` into this new file:

```
import "fmt"

type Book struct {
    Title  string
    Author string
    Copies int
}

func BookToString(book Book) string {
    return fmt.Sprintf("%v by %v (copies: %v)",
        book.Title, book.Author, book.Copies)
}
```

(Listing books/10)

We included an `import "fmt"` declaration, too, because we need to use that in `BookToString`.

Updating the test

Next, let's rename the `main_test.go` file to `books_test.go`, since it's about testing the `books` package. We'll change its first line to:

```
package books_test
```

Now, we need to make a few changes to the test. Because we'll be using stuff from the `books` package, we'll need to import it, just like any other package we use such as `testing`:

```
import "books"
```

And it's no longer enough for us to simply mention a struct type like `Book` in our test code. That type belongs to another package, so we need to *qualify* its name by putting the package name first, just like we do with `fmt.Println`:

```
input := books.Book{
    Title:  "Sea Room",
    Author: "Adam Nicolson",
    Copies: 2,
}
```

```
...
got := books.BookToString(input)
```

In other words, in this test we're doing exactly what anyone else would have to do if they wanted to use our bookstore code: import the package, and then call our functions using their full name, like `books.BookToString`. If it works for us, it should work for them, too!

So here's the complete, updated `books_test.go` file:

```
package books_test

import (
    "books"
    "testing"
)

func TestBookToString_FormatsBookInfoAsString(t *testing.T) {
    input := books.Book{
        Title:  "Sea Room",
        Author: "Adam Nicolson",
        Copies: 2,
    }
    want := "Sea Room by Adam Nicolson (copies: 2)"
    got := books.BookToString(input)
    if want != got {
        t.Fatalf("want %q, got %q", want, got)
    }
}
```

(Listing `books/10`)

Notice that we can import both the `books` and `testing` packages in a single `import` declaration, by surrounding them with parentheses. This is neater than repeating `import` on each line.

Updating the `main` package

Next, we'll make the same changes to our `main` package. We'll need to update `main.go` to import the `books` package and use the full names of its functions and types:

```
package main

import (
    "books"
    "fmt"
)

func main() {
    book := books.Book{
        Title:  "Engineering in Plain Sight",
        Author: "Grady Hillhouse",
        Copies: 2,
    }
    fmt.Println(books.BookToString(book))
}
```

(Listing books/10)

This is a completely reasonable thing to write, but all the same, if we try to run it, Go has a complaint:

```
found packages books (books.go) and main (main.go) ...
```

What this means is that Go doesn't allow two different packages to cohabit in the same folder. We can *have* as many packages as we want in our project, so long as they live in different folders. So either `books` or `main` has to move somewhere else. Which do you think it should be?

Creating a `cmd` subfolder

It makes sense to me that the `books` package should be in the top-level folder, at the "root" of our project, since books are fundamentally what this project is about.

In that case, then, if we want to add a new `main` package for a command that users can run, we'll need to create a subfolder for it. We can call this whatever we want, but `cmd` (for "command") is the convention most Go programmers use:

```
mkdir cmd
```

And, because (spoiler alert), we're going to end up having more than one command eventually, let's create a further subfolder for *this* command. Since it currently lists all the books we have in our catalog, let's name it `list`:

```
mkdir cmd/list
```

And now we can move the offending `main.go` file into this subfolder:

```
mv main.go cmd/list
```

Checking the results

Now that we've resolved the "clashing packages" issue, we should be able to run the tests:

```
go test
```

```
PASS
ok      books   0.226s
```

And we should also be able to run our `list` command:

```
go run ./cmd/list
```

```
Engineering in Plain Sight by Grady Hillhouse (copies: 2)
```

Great job! We're well on our way to a thriving bookstore. We have a package that can store and format information about books, and we have a command that sets up a catalog containing a single book, and prints its details.

But customers can be rather demanding, and they might want us to be able to stock more than a single book. That means updating our program to handle *multiple* books at once, and we'll see how to do that in the next chapter.

Here's what we know so far

You've learned several new key ideas in this chapter: all the techniques we need to write *self-testing code*, including conditional statements, comparison operators, `panic`, `return`, formatting data with `fmt.Sprintf`, and Go's built-in `testing` package.

You also now know how to write useful test names, and how to view them as docs using `gotestdox`, as well as how to write informative failure messages. You're becoming more familiar with the "guided by tests" workflow we use for adding new features, fixing bugs, or changing the behaviour of existing functions.

Usefully, you've learned how to put your code in an importable package, and how to create a separate `main.go` file that imports the package and uses it to do something.

4. Slicing & dicing

You've made a great start on our bookstore project so far, but there's something we're still missing. We can create a single value of the `Book` type and assign it to a variable, but a decent bookstore is going to need more than one book. We don't yet have a way of dealing with *collections* of things (books, for example).

Slices

Just as we grouped a bunch of values of different types together into a struct, so that we can deal with them as a unit, we would like a way to group together a bunch of values of the *same* type. And in Go one kind of collection of values is called a *slice*.

For example, a value representing a group of books would be a *slice of `Book`* (that's how we say it). We write it using empty square brackets before the element type: `[]Book`. Just as each individual `struct` type in Go is a distinct type, so is each slice.

Slice variables

Can we declare variables of a slice type? Yes, we can:

```
var books []Book
```

This says "Please create a variable named `books` of type `[]Book`" (slice of `Book`). Next, let's assign something to it.

Slice literals

We've seen how to write struct literals; now here's how to write a *slice literal* of structs:

```
books = []Book{}
```

The value on the right-hand side of the assignment is an empty `[]Book` literal. If we wanted to include some actual books in it, we could do that, by putting some `Book` literals (each with a trailing comma) inside the curly braces:

```
books = []Book{
    {
        Title:  "Chasing the Thrill",
        Author: "Daniel Barbarisi",
    },
    {
        Title:  "Birds and People",
        Author: "Mark Cocker",
    },
}
```

Notice that we don't need to write the type name `Book` before each struct literal inside the slice, because Go knows that only `Book` values can possibly be members of a slice of `Book`.

Indexing slices

There's something we can do with a slice that we couldn't do with a basic or struct type: we can refer to one of its *elements*. If we just want, for example, the first book in the slice, we can specify which element we want using its *index* number in square brackets:

```
first := books[0]
```

The index is a non-negative integer that specifies the single element we're interested in, where the index of the first element is 0, the second is 1, and so forth. (Think of the index as an *offset* from the start of the slice, if that helps.)

You might be wondering what happens if we ask Go for an element that doesn't exist. For example, our `books` slice has two elements, whose index numbers are 0 and 1. So

what would we get if we asked for index number 2? That doesn't correspond to any element in the slice, so let's try:

```
fmt.Println(books[2])
```

Oh dear:

```
panic: runtime error: index out of range [2] with length 2
```

Well, we expected *some* error, and here it is. First, note that the program *panics*: we've encountered this before when we used the built-in function `panic` to fail our homemade test for `BookToString`.

This time, though, the program panics without us asking it to. This is because of a *runtime error*: although the program compiles successfully, it doesn't work when we run it. And we already know the specific error in this case, because we caused it accidentally-on-purpose to see what would happen:

```
index out of range [2] with length 2
```

In other words, Go is telling us, you can't ask for the third element of a slice with only two elements. That *must* be a mistake on the part of the programmer, so the sensible thing for Go to do is stop the program and wait for us to correct it.

Finding the length of a slice

We really don't want our programs to panic, as a rule, because that would mean they have a bug. In this case, the bug is that we recklessly indexed a slice without first checking that it has the required number of elements.

Given a slice variable of a certain length, then, how can we tell which indexes are safe to use? That's equivalent to knowing how many elements are in the slice, and there's a built-in function `len` (short for "length") for that:

```
fmt.Println(len(books))
// Output:
// 2
```

This is telling us that the `books` slice has 2 elements. Any time we want to request a certain slice element by its index, then, it's wise to check the length of the slice first using `len`, to make sure that the program won't panic.

Modifying slice elements

Can we modify an individual slice element directly, using its index? It turns out we can:

```
books[0] = Book{
    Title:  "The Power Broker",
    Author: "Robert A. Caro",
}
```

Indeed, we can use the same trick as with structs, to modify an individual *field* of an individual element:

```
books[0].Title = "The Power Broker"
```

Appending to slices

There's something else we can do with a slice that we couldn't do with our other types: we can *add a new element* to it. We use the built-in `append` function for this:

```
newBook := Book{
    Title:  "Skyjack",
    Author: "Geoffrey Gray",
}
books = append(books, newBook)
```

`append` takes a slice and an element value (or multiple values), and returns the modified slice, with the new value as its last element.

A collection of books

Let's implement another feature for the bookstore: listing all books. Suppose we implement this with a function called `GetAllBooks`. How shall we write a test for it? What's the behaviour we want?

Well, we need some concept of "the collection of books in stock". We can now make a guess that a *slice* of books will be a useful way to represent this collection. How do we spell that in Go? Its type will be `[]Book`.

So there needs to be some `[]Book` variable that we will use to store the books we have in stock: our *catalog*. Let's call this variable `catalog`, then: simple and obvious.

We're starting to put together the pieces we need: a slice variable `catalog`, and a function `GetAllBooks`. Whatever is contained in `catalog`, that's what `GetAllBooks` should return. Let's start sketching out a test along those lines.

Setting up the world

We know we'll need to call `GetAllBooks` in the test, and check its return value against some expectation, just like we did with the test for `BookToString`.

What's our `want`? Well, we expect `GetAllBooks` to return a slice containing all the books in our catalog, so that's our `want`. We can make the catalog as large or small as we want for this test, but it makes sense to keep it fairly small, to save typing. Let's say it contains just two books:

```
want := []books.Book{
    {
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
    },
    {
        Title:  "White Heat",
        Author: "Dominic Sandbrook",
        Copies: 2,
    },
}
```

Having set up this expected value, we can now call the function to get the *actual* value:

```
got := books.GetAllBooks()
```

Finally, we're ready to compare `want` with `got` to see if the function returned the correct result.

Comparing slices

There's just one problem. Usually in a test we compare `want` with `got`, to see if it was as expected. But we can't actually compare two slices in Go directly, using the `==` or `!=` operators:

```
if want != got {
// invalid operation: want != got (slice can only be compared
// to nil)
```

Instead, we can use the standard library function `slices.Equal` to do this comparison for us. It takes two slices, and tells us if they're the same. Of course, we want to fail the test if the two slices are *not* equal, so here's how we write that:

```
if !slices.Equal(want, got) {
```

Notice the `!` in front of `slices.Equal`; it's pronounced "not", in the same way that the `!=` operator is pronounced "not equal to". In general, any condition in an `if` statement can be inverted (flipped from true to false, or vice versa) by putting the `!` operator in front of it like this.

The updated test

Here's the complete test, ready to run:

```
func TestGetAllBooks_ReturnsAllBooks(t *testing.T) {
    want := []books.Book{
        {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
        },
        {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
        },
    }
    got := books.GetAllBooks()
    if !slices.Equal(want, got) {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}
```

(Listing `books/11`)

Here's another new format placeholder: instead of `%v`, we've used `%#v`. This prints its corresponding data as a Go value—in this case, it'll be a slice of `Book` structs.

A dummy `GetAllBooks`

It's not compiling yet, of course, because the `GetAllBooks` function doesn't exist, but we know what to do about that. Just as we did with the test for `BookToString`, we'll write a dummy version of the function that does basically nothing, and we'll use that to check that the *test* works.

GOAL: Add the minimum code necessary to get this test to compile and fail.

HINT: Well, we know we need a new function. The test makes it clear what the name of this function should be, and what parameters (if any) it takes. Because we'll be comparing its result with our `want` variable, the function must return the same type as `want`: that is, `[]Book`.

Can you see what to do?

SOLUTION: Here's the simplest incorrect version of `GetAllBooks` that I can think of:

```
func GetAllBooks() []Book {
    return []Book{}
}
```

It returns an *empty* slice of `Book`, which definitely isn't the same as `want`, so the test should fail, and indeed it does:

```
--- FAIL: TestGetAllBooks (0.00s)
    books_test.go:24: want []books.Book{books.Book{Title:"In the
    Company of Cheerful Ladies", Author:"Alexander McCall Smith",
    Copies:1}, books.Book{Title:"White Heat", Author:"Dominic
    Sandbrook", Copies:2}}, got []books.Book{}
```

We can see that the `%#v` format prints the `want` slice using Go syntax, and it contains what we expect. The `got` slice is empty, also as we expect, and that makes the test fail, so everything's working as it should. Great. Let's go ahead and make the test pass now.

Implementing `GetAllBooks`

GOAL: Make this test pass.

HINT: We already have the "skeleton" version of `GetAllBooks`, which returns an empty slice. All you need to do is change it to return a slice with some elements. And you know what the elements should be: they're the same as the ones in our `want` variable in the test. See what you can do.

SOLUTION: Here's one way to do it:

```
var catalog = []Book{
    {
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
    },
    {
        Title:  "White Heat",
        Author: "Dominic Sandbrook",
        Copies: 2,
    },
}

func GetAllBooks() []Book {
    return catalog
}
```

(Listing books/11)

This passes, and we can also use `gotestdox` to remind us what our test asserts about the function's behaviour:

```
gotestdox
```

```
✔ GetAllBooks returns all books (0.00s)
```

Because of course it does! Another good choice of name, I think; there's a lot to be said for obviousness-oriented programming.

A global `catalog` variable

If your solution is different, that's okay. If it passes the test, that's all we need for the moment. For example, you could have defined the `catalog` variable *inside* the `GetAllBooks` function, instead of outside. Or you could even have dispensed with the variable altogether, and simply returned the slice as a literal. That's fine.

So why have I written it this way? I'm thinking ahead a little bit, because we know that in the real bookshop our catalog of books won't always be a fixed value. It will *vary* over time, as we add and remove books from stock, so it makes sense to store the books in a variable.

And because we'll probably want to write functions to add and remove books, they'll also need access to this variable. If we defined `catalog` in the body of the `GetAllBooks` function, then no other function would be able to see it: it would be *local* to that function.

Up to now, all the variables we've defined have been local ones. Their *scope*—the part of the program that's allowed to refer to them—has been confined to the function body they live in. But, by moving the `var` declaration outside the function, we can also create variables whose scope is the whole package. That makes them visible to any function within the package, and that'll be useful to us later on.

Variables like this are sometimes loosely called "global", by contrast with "local". But since Go programs can be made up of multiple packages, it's not really correct to say that `catalog` is a global variable. To be accurate, we should call it "package-scoped": it's only visible within the package where it's defined.

Rewriting `list`

Now that we've expanded the capabilities of our bookstore to stock more than one book, let's try it out! Suppose we use our new `GetAllBooks` function to write a new version of the `list` program that users can run to see the complete catalog of books in stock.

We could start with something like this in `cmd/list/main.go`:

```
func main() {
    fmt.Println(books.GetAllBooks())
}
```

Let's see what it looks like:

```
go run ./cmd/list
```

```
[{In the Company of Cheerful Ladies Alexander McCall Smith 1}
{White Heat Dominic Sandbrook 2}]
```

Mmm. Could be better. Of course, this kind of output is fine for us programmers, when we're testing the code. But real users would like something a bit more friendly.

Improving the output format

Actually, we already have a nice function for formatting a book's details as a user-readable string: it's `BookToString`. Now we can start getting some wins from the abstraction-based approach, by plugging this component together with the "get all books" component.

Here's a first attempt:

```
func main() {
    fmt.Println(books.BookToString(books.GetAllBooks()))
}
```

But this doesn't compile:

```
go run ./cmd/list
```

```
cannot use books.GetAllBooks() (value of type []books.Book) as
books.Book value in argument to books.BookToString
```

Okay, that sort of makes sense. We defined `BookToString` to take a *single* `Book` as its argument. But what we're *passing* here is what `GetAllBooks` returns, which is a *slice* of books, that is, `[]Book`. That's a different type to `Book`, so Go won't accept it.

How can we fix this mismatch? We *could* change `BookToString` to accept a slice of books, and format them all. But that probably wouldn't be the best idea, because then if we wanted to print the details of just one book, we'd have the opposite problem!

A `for ... range` loop

Instead, what we really want to do is take each book one at a time, use `BookToString` to format its details, and print the result. If we repeat this process for every book in the catalog, it'll produce the output we need.

A program construct that repeats the same bit of code several times is called a *loop*, and in Go we use the `for` keyword for that. When we combine it with the `range` operator, we get a loop that executes once for each element of a slice. Perfect!

```
func main() {
    for _, book := range books.GetAllBooks() {
        fmt.Println(books.BookToString(book))
    }
}
```

(Listing `books/11`)

The blank identifier

There's just one thing that looks a bit puzzling about this `for ... range` construct. We can see that, each time we enter the loop, `book` is the next book in the slice. That makes sense: we're executing some code "for each book", if you like. But what's this mysterious `_`?

```
for _, book := range...
```

The answer is that the `range` operator gives us *two* values, not one, for each slice element. The second, `book`, is the element itself, just as you'd expect. But the first will be the *index* of that element: these go 0, 1, 2..., as we saw earlier in this chapter.

As it happens, we don't use the index numbers in this program, so we'll just ignore them, by putting _ in the place where a variable name would go (_ is called the *blank identifier*: it means "I don't need this, thanks").

Let's try out the updated program:

```
go run ./cmd/list
```

```
In the Company of Cheerful Ladies by Alexander McCall Smith
(copies: 1)
White Heat by Dominic Sandbrook (copies: 2)
```

Very nice!

Unique identifiers

We've added some useful functionality for our prospective bookstore colleagues: they can run a program and print out the complete catalog of books in stock. This is great as far as it goes, but we can already see some limitations.

Finding a specific book

For example, suppose a customer asks if we have a certain book in stock. The program we have *could* answer that question, but the user would have to print out the whole catalog and then read through it, book by book, until they eventually find the book the customer wants. Or, if they can't, they'll know it's probably not in stock.

That's rather laborious and error-prone work, which is the sort of thing that computers are supposed to help us avoid. What we'd like here is the ability to find a *specific* book in the catalog, if it exists, and see the details.

That sounds like a function we could write, doesn't it? Let's call it `GetBook`. We could call it, specifying the book we want, and it could return the corresponding `Book` struct. Then we could print it out, using `BookToString`, for example.

Specifying a book uniquely

So, how do we "specify" a book? The customer might know the title and the author, or just the title, or just the author. Or they might not know either of those things, and the task of finding what they want might be even harder: "You know, the one with the boy wizard, not Harry Potter, the other one".

In a real bookstore, it would be useful to search the catalog by any property of the book, such as author or title, and maybe even do "fuzzy" searches, such as finding all books with "vampire" in the title.

Let's take things one step at a time, though, and start with a very simple search facility: we'll assume each book has some kind of unique identifier, and when we pass that to `GetBook`, we'll either find the corresponding book, or we'll know that it's not in the catalog.

How *do* we uniquely identify a book, then? Not by its title, because there can be many books with the same title. Nor by its author, price, or any of the other fields on `Book`, either, because none of them are unique.

There is such a thing as an International Standard Book Number (ISBN), but not every book has one; the one you're reading doesn't, for example! (They're expensive and, for self-published books, unnecessary.)

So we're going to need some unique identifier (ID) for each book. To be clear, it doesn't matter what that value actually *is* for any given book, only that it's unique within our catalog. If the book already has an ISBN, we could use that, or we could assign our own IDs to books that don't have them.

What type would make sense for the `ID` field, then? Well, `int` wouldn't work, because an ISBN contains letters as well as numbers. In general, we can't know in advance what format the ID might take. So the safest choice for this field is `string`, because a string can hold letters, numbers, or any other characters defined by Unicode.

Designing `GetBook` to take a book ID

Now we can start to see how to sketch out a test for `GetBook`. It would take a string parameter representing the book ID, which can be whatever we want for the purposes of the test ("abc", for example). Assuming there *is* a book in our catalog with such an ID, which we'll need to arrange, then the test can check that `GetBook` returns exactly that book.

Over to you to have a go at this, then!

GOAL: Write a test for `GetBook` along the suggested lines, making the necessary changes to the rest of the program to get it to compile, and the test to fail. Remember, you don't need to implement the real version of `GetBook` yet, only the test, and a stub version of `GetBook` that returns the wrong answer.

HINT: You know how to write a test, how to add a new field to a struct type, and so on. See how far you can get before looking at the suggested solution.

SOLUTION: Here's what I came up with:

```
func TestGetBook_FindsBookInCatalogByID(t *testing.T) {
    t.Parallel()
    want := books.Book{
        ID:     "abc",
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
    }
    got := books.GetBook("abc")
    if want != got {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}
```

Parallel tests

There's one extra new feature that I sneaked in: the `t.Parallel()`. This tells Go that it's okay to run this test in *parallel* with others: that is, at the same time. If you have more than one CPU in your computer, which you almost certainly do, then Go can put them all to work running tests concurrently, which makes sense.

We'll add `t.Parallel()` to our other tests, too. There's no reason *not* to run tests in parallel, and in general it's a good idea: it saves time, and when you have a lot of tests, the saving can be quite significant.

This test isn't too complicated in itself. The name makes sense when seen through `gotestdox`:

```
GetBook finds book in catalog by ID
```

We assert that if the book "In the Company of Cheerful Ladies" has the arbitrary ID `abc`, then when we call `GetBook` with that ID, it should return the book we're looking for.

Adding the `ID` field to `Book`

So far, so good, but to get the program to even compile needs a couple of changes. We've referenced a field `ID` on `Book` as though it existed, but it doesn't yet. I'm sure you figured out how to add it:

```
type Book struct {
    Title  string
    Author string
    Copies int
    ID     string
}
```

(Listing books/12)

The order of the fields doesn't matter for our program, so you could put the `ID` field first, last, or anywhere you like.

A dummy `GetBook`

We still have a compile error because no function `GetBook` exists, but you know how to deal with that. We can write a very simple stub version of `GetBook`:

```
func GetBook(ID string) Book {
    return Book{}
}
```

It doesn't actually matter *what* we return for now, so long as it's the wrong answer, and a completely empty `Book` struct will certainly be that. In fact, as you now know, variables always have default values in Go, so a `Book` where we don't explicitly set any of the fields will nonetheless contain the default values for each of those fields.

The program now compiles, and the test fails in the way we expect:

```
--- FAIL: TestGetBook_FindsBookInCatalogByID (0.00s)
    books_test.go:40: want books.Book{Title:"In the Company of
    Cheerful Ladies", Author:"Alexander McCall Smith", Copies:1,
    ID:"abc"}, got books.Book{Title:"", Author:"", Copies:0, ID:""}
```

Nice work! So now let's see if we can make the test pass, by writing the real version of `GetBook`.

What does `GetBook` need to do?

This isn't actually all that difficult, and it doesn't require any knowledge that you don't already have. What you might find challenging at first is just figuring out *what to do*.

In other words, if you had an *algorithm*—an English description of the steps the function needs to take—you could probably work out how to express that in Go code. But what's our algorithm here?

One trick I use in a situation like this is to ask myself, "What would I do *manually*?" That is to say, if I were looking for a particular book among a bunch of real, physical books, what would I do?

I can't rely on the books being in any particular order—that would be great, but we don't have that luxury in the program, so I wouldn't in real life. Instead, I'd simply have to look at each book in turn, asking myself "Is this book's ID the one I'm looking for?" When I find the one with the matching ID, then I'm done.

Implementing the algorithm

Let's see if we can write this procedure in Go. First, can we figure out how to look at each book in the catalog in turn? Yes, we know how to do this already, using `for ... range`, as we did in Listing `books/11`:

```
for _, book := range catalog {
```

And we also know how to use an `if` statement to do something *if* some condition is true. Here, the condition is that the ID of this book is the one we're looking for:

```
if book.ID == ID {
    return book
}
```

If this is the book we want, in other words, then return it—we're done! No need to keep on searching through the remaining books, since we've found what we were looking for.

A first attempt at `GetBook`

So here's our first sketch:

```
func GetBook(ID string) Book {
    for _, book := range catalog {
        if book.ID == ID {
            return book
        }
    }
}
```

But there's a problem: this doesn't compile. Go complains, at the end of the function:

```
missing return
```

That's because the function signature declares that it returns a `Book`. But, looking at this code, we can see that it might not! If we loop over all the books there are, and none of them match the requested ID... then what?

Handling the "not found" case

We have to return *something*, because the function promises that it will. And it has to be a `Book`. If we don't *have* the requested book, though, what book should we return? It's not obvious.

Returning multiple results

We could use an empty `Book` struct, as we did in the stub version, but that's annoying for users. Every time they called `GetBook`, they'd have to look at the ID of the returned book to see if it matched the ID they asked for. If it didn't, that would mean, effectively, "book not found".

That's more paperwork than users want to do. As the caller of this function, what we'd *like* is a simple yes-or-no indicator, telling us whether the book was found or not. That's not impossible, but the problem is that we already declared the function to return a result of type `Book`, which isn't a yes-or-no data type. What can we do?

We can return *two* results from the function! This is a neat feature of Go which isn't available in every other programming language, and it comes in handy for cases like this. And Go also has exactly the data type we need: it's called `bool`, short for "Boolean".

Boolean values

Not the most intuitive name, perhaps, but it's named after the mathematician George Boole, who was one of the first to explore the idea of binary numbers (in effect, values that can be "yes" or "no"). As you can imagine, that's a pretty fundamental concept in computing, so Boolean values are used a lot.

It doesn't actually matter what we choose to call the two possible values of a `bool`: you can think of them as "yes" or "no", 0 or 1, "true" or "false", and so on. The point is that they can only be one or the other.

In Go the possible values are called `true` and `false`. Just as with the other data types we've met already, we can declare a variable of type `bool` and assign it one of the possible values:

```
var bookFound bool
bookFound = true
bookFound = false
```

And we can also take `bool` parameters to functions, or have them return `bool` results. That's just what we need here, in fact: a result value from `GetBook` that means "found" or "not found".

Returning a `bool` result from `GetBook`

Let's update `GetBook` to return a `bool` along with the `Book`, then:

```
func GetBook(ID string) (Book, bool) {
```

Notice that now we have to put parentheses around the list of results, just as we do with the list of parameters. If there were only one result, we could leave out the parentheses, but as soon as we have two or more results, the parentheses are required.

So this extra result will be a signal to callers: "Yes, I've found the book you wanted", or "Sorry, we don't have that book in stock", depending on what happens in the search. If we find a match for the requested ID, then it makes sense to return `true` along with the matching book:

```
if book.ID == ID {
    return book, true
}
```

The key point is that now we know what to return at the *end* of the function, if we've completed the entire loop without finding the requested book. We can return an empty `Book`, because there's no other `Book` value that makes sense, but we can *also* return `false`, meaning "not found".

So here's the updated function:

```
func GetBook(ID string) (Book, bool) {
    for _, book := range catalog {
        if book.ID == ID {
            return book, true
        }
    }
    return Book{}, false
}
```

(Listing `books/12`)

This makes it really easy for users to check whether `GetBook` found the book or not.

Testing the ok value

Let's update the test to check this second result from `GetBook`:

```
got, ok := GetBook("abc")
if !ok {
    t.Fatal("book not found")
}
```

Seems reasonable, doesn't it? We still have an assignment statement, just as before, but now because `GetBook` returns *two* values, we need two variables on the left hand side: we'll call them `got` and `ok`.

They're in the same order as the results `GetBook` returns, naturally enough: first the book, then the `bool` value meaning "found or not found". We could have called the second variable something like `found`, but it's conventional in Go to call it `ok`.

You can think of it as indicating "Okay, I found what you wanted, and it's in the other result". Or, alternatively, "Sorry, I couldn't find what you wanted, so don't bother looking at the other result—it'll just be empty."

The happy path

Which answer do we expect in the test? Well, that's a good question. In *this* test, by definition we expect `ok` to be `true`, meaning that we *did* find the book whose ID is "abc". That's why we write:

```
if !ok {
```

Remember how we used `!` earlier to mean "not", as in `!slices.Equal`? It's the same thing here. `if !ok` means "if `ok` is false" (that is, if the book was not found) then the test should fail.

Here's what we have so far:

```
func TestGetBook_FindsBookInCatalogByID(t *testing.T) {
    t.Parallel()
    want := books.Book{
        ID:     "abc",
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
    }
    got, ok := books.GetBook("abc")
```

```
    if !ok {
        t.Fatal("book not found")
    }
    if want != got {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}
```

(Listing books/12)

In other words, if we search for a book that *is* in the catalog, we should find it.

Programmers often call this kind of behaviour the "happy path", meaning what's supposed to happen if everything goes according to plan. But, since that isn't always the case, we also need to account for the "sad path".

The sad path

That is, if we search for a book that's *not* in the catalog, we *shouldn't* find anything. You might think that's not worth testing, but it is. For example, `GetBook` might accidentally always return true for `ok`, whether or not the book exists, and I think we agree that would be wrong.

So let's test that when the book is not found, the `ok` value returned is `false`:

```
func TestGetBook_ReturnsFalseWhenBookNotFound(t *testing.T) {
    t.Parallel()
    _, ok := books.GetBook("nonexistent ID")
    if ok {
        t.Fatal("want false for nonexistent ID, got true")
    }
}
```

(Listing books/12)

There's no `want` this time, because we're not *expecting* to find the book, so we don't need to compare it with anything. Indeed, we don't need the variable `got` either, so we use the blank identifier `_` to ignore the first value returned by `GetBook`.

The only thing we're interested in for *this* test is the `ok` value. Since we expect it to be `false` for a nonexistent book, we fail the test if it isn't. Seems reasonable to me!

Adding IDs to the catalog

We can now run these tests and see what happens. Well, the "book not found" test passes, which is encouraging. The "book found" test doesn't, though, and the reason isn't hard to figure out. We're searching for a book with a specific ID, and the books in our catalog don't actually *have* IDs yet.

Or rather, since struct fields always have a default value, every book's `ID` field is the empty string. Let's fix that:

```
var catalog = []Book{
    {
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
        ID:     "abc",
    },
    {
        Title:  "White Heat",
        Author: "Dominic Sandbrook",
        Copies: 2,
        ID:     "xyz",
    },
}
```

(Listing `books/12`)

We're saying it doesn't matter *what* ID we assign to each book, as long as they're all different (and correspond to the ones we expect in the tests).

Fixing the tests

That fixes the "book found" test, but now our *previous* test, for `GetAllBooks`, is failing:

```
--- FAIL: TestGetAllBooks_ReturnsAllBooks (0.00s)
    books_test.go:26: want []books.Book{books.Book{Title:"In the
    Company of Cheerful Ladies", Author:"Alexander McCall Smith",
    Copies:1, ID:""}, books.Book{Title:"White Heat",
    Author:"Dominic Sandbrook", Copies:2, ID:""}}, got
    []books.Book{books.Book{Title:"In the Company of Cheerful
    Ladies", Author:"Alexander McCall Smith", Copies:1, ID:"abc"},
    books.Book{Title:"White Heat", Author:"Dominic Sandbrook",
    Copies:2, ID:"xyz"}}
```

Not to worry. This is the sort of thing we deal with all the time when making changes

to programs. The changes are bound to break *some* tests, which is a good sign: it shows we're testing the code thoroughly.

And in this case the code for `GetAllBooks` is still perfectly correct; it's the *test* that needs updating. Books now have IDs! Once we update the `want` value to allow for that, everything is fine:

```
func TestGetAllBooks_ReturnsAllBooks(t *testing.T) {
    t.Parallel()
    want := []books.Book{
        {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
            ID:     "abc",
        },
        {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
            ID:     "xyz",
        },
    }
    got := books.GetAllBooks()
    if !slices.Equal(want, got) {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}
```

(Listing books/12)

Great job! We can list all books *and* search by ID. Or rather, we have the facility to search by ID, but we don't yet have a program that Happy Fun Books operatives could run to actually search for something. Let's tackle that problem in the next chapter.

Here's what we know so far

You've covered a lot of ground in this chapter, and we're really getting to grips now with data handling in Go. You've learned how to manage collections of values by storing them in a slice, how to declare slice variables and write slice literals, how to read and write individual slice elements, how to append new elements to a slice, and how to get the length of a slice using the `len` function.

You've also learned how to process each element of a slice in turn using a `for ... range` loop, and how to compare two slices with `slices.Equal`, which is very useful for tests. And you've learned about the `bool` type, and how to return it from a function to signal whether or not some result is present. Nice work!

5. Map mischief

Thanks to your excellent work in the previous chapter, we now have the code necessary to search for a book by its ID, and retrieve its details. This is the sort of thing that we'll need for Happy Fun Books, so let's write a program that staff could run to do a book search.

A `find` command

First, we need to create a new subfolder in `cmd` for the command we'll be adding. Shall we call it `find`?

```
mkdir cmd/find
```

This is where our new `main.go` file will live. So let's start with a quick sketch.

Looking up a book by ID

We know how to call `GetBook`, because we've already done it in a test. And it takes the ID of the book we're looking for, so where do we get that from? In practice, users will want to supply the ID on the command line, but let's keep things simple for now and start by looking up some fixed ID—let's say `xyz`.

```
got, ok := books.GetBook("xyz")
```

And we know that the `ok` variable tells us whether the book was actually found. If it's not, we should probably inform the user:

```
if !ok {
    fmt.Println("Sorry, I couldn't find that book in the catalog.")
    return
}
```

How polite! Courtesy costs nothing, after all. If the book *is* found, though, we should print out its details, and we know how to do that, too:

```
fmt.Println(books.BookToString(book))
```

Implementing `find`

The pieces are coming together. So here's what we have:

```
package main

import (
    "books"
    "fmt"
)

func main() {
    book, ok := books.GetBook("xyz")
    if !ok {
        fmt.Println("Sorry, I couldn't find that book in the catalog.")
        return
    }
    fmt.Println(books.BookToString(book))
}
```

Let's run it and see what happens:

```
go run ./cmd/find
```

```
White Heat by Dominic Sandbrook (copies: 2)
```

Great. But we need to be able to search for *any* book by ID, not just this one. That means we'll need to be able to supply the ID as an *argument* to the program. For example, users might run a command like:

```
go run ./cmd/find abc
```

Command-line arguments and `os.Args`

The standard library has a variable `os.Args` that we can use to get the command-line arguments. When the program starts, the value of `os.Args` is a `[]string`, where the first element of the slice (index 0) is the program name itself, index 1 is the first argument, and so on.

Fine. So if the user gives the book ID as an argument, we will find it in `os.Args[1]`. Let's see what we can do:

```
package main

import (
    "books"
    "fmt"
    "os"
)

func main() {
    ID := os.Args[1]
    book, ok := books.GetBook(ID)
    if !ok {
        fmt.Println("Sorry, I couldn't find that book in the catalog.")
        return
    }
    fmt.Println(books.BookToString(book))
}
```

This looks reasonable, so here goes:

```
go run ./cmd/find abc
```

```
In the Company of Cheerful Ladies by Alexander McCall Smith (copies: 1)
```

Checking arguments

This is encouraging, but now that the program *requires* an argument, what will happen if we run it without one?

```
go run ./cmd/find
```

```
panic: runtime error: index out of range [1] with length 1
```

Whoops! This makes sense under the circumstances: as we saw in the previous chapter, asking for a slice element that doesn't exist is a run-time error. But the message isn't very friendly. Let's instead check to see if we have an argument, and if not, show a more useful response:

```
func main() {
    if len(os.Args) != 2 {
        fmt.Println("Usage: find <BOOK ID>")
        return
    }
    ID := os.Args[1]
    book, ok := books.GetBook(ID)
    if !ok {
        fmt.Println("Sorry, I couldn't find that book in the catalog.")
        return
    }
    fmt.Println(books.BookToString(book))
}
```

(Listing books/13)

Now we not only don't panic, but we also show the user how to use the program correctly. Much better.

A more efficient data structure

So we've made good progress towards supporting Happy Fun Books operations, but there's still something that doesn't seem quite right about the way we've implemented `GetBook` here. Given a book ID, we search for it in the catalog by simply looking at every book in turn, asking, in effect, "Is this the one?"

Clearly this *works*, in that it's guaranteed to find the book eventually if it's in the catalog. But, with a large catalog, "eventually" could be quite a long time. We might be lucky and find the desired book near the start of the catalog, or we might be unlucky and only find it near the end.

Big-O notation

On average, if we have N books, we'll have to check $N/2$ of them before we find the one we want. That means the more books we have, the slower it becomes to find any

one of them. Surely we can do better than that?

In computer science, there's a way to describe this behaviour of an algorithm as the number of items scales up: it's called *big-O notation*. If the time to search the catalog grows linearly with the number of books N, as it does in our current implementation, we say that our algorithm is $O(N)$, or "order N". The bigger N is, the longer the search takes (on average).

Maybe a slice isn't the best way to store our book catalog, then, since a time-consuming linear search is the only way to find something in it. Ideally, we want a data structure that, given a book ID, returns the corresponding book directly. In other words, it has a direct *mapping* from IDs to books.

This would be an $O(1)$ algorithm, or "order 1", meaning that the search time never gets any longer, no matter how big N is: the scaling factor is 1. That sounds great! Can we do it?

Introducing the map

It turns out that Go has exactly the data type we want. And, not surprisingly under the circumstances, it's called a *map*.

A map is quite similar to a slice, in that it's a collection of elements, all of the same type. The difference is in how we add and retrieve those elements.

With a slice, to get a specific element, we can reference its *index* number, giving its position in the slice. That is to say, each element effectively has a unique ID that's some positive integer.

A map, though, can use any Go type as the unique ID: what's called the *key*. For example, we could use a string, like our book ID. The nice thing about a map is that we don't have to search through every element in turn. Given a key, the map will return the corresponding element straight away (if it's there).

This means that, no matter how large our catalog of books, book searches will always take the same amount of time. That sounds perfect, so let's see if we can update the program to use a map instead of a slice for the catalog.

Choosing the map type

First of all, do we need to update the tests? Surprisingly, perhaps, we don't. The only two functions affected are `GetBook` and `GetAllBooks`, and while we'll need to change their implementation, their *behaviour* won't change. (At least, not in a way that's visible to the tests; they'll probably be faster, though, especially with real data).

The most important change we'll need to make is to the definition of the `catalog` variable: the central repository of books we have in stock. Right now, it's defined as a slice of `Book`, and we need to change that to a map instead.

As you'll remember from the previous chapter, there's no such type as "a slice" in Go, only a slice of some specific type such as `Book`. Similarly, there's no such type as "a map", only a map of some type (the key) *to* some other type (the element).

What do we need here? Our key will be a `string`, like "abc". The element, just as before, will be a `Book`. That type in Go is spelt `map[string]Book`. It's all one word, no spaces, with `map` followed by the key type in square brackets, followed by the element type.

Map literals

The syntax for a map literal of this type is pretty straightforward:

```
var catalog = map[string]Book{
    "abc": {
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
        ID:     "abc",
    },
    "xyz": {
        Title:  "White Heat",
        Author: "Dominic Sandbrook",
        Copies: 2,
        ID:     "xyz",
    },
}
```

(Listing books/13)

It's quite similar to the corresponding slice literal, isn't it? The only difference is that instead of a comma-separated list of elements, we have a comma-separated list of *key-element pairs*:

```
"key": element,
...
```

Turning maps into slices

Okay. Since we've changed the type of `catalog`, we expect to also have to make some changes to the functions that access the catalog: `GetBook` and `GetAllBooks`. Let's check `GetAllBooks` first. Do we have any compile errors?

Yes, we do. Here's the code we have right now:

```
func GetAllBooks() []Book {
    return catalog
}
```

When `catalog` itself was defined as a `[]Book`, this was fine. Essentially, `GetAllBooks` has no real work to do: it just returns `catalog` directly.

This won't do now that we've changed `catalog` to be a map: we get a type mismatch.

```
cannot use catalog (variable of type map[string]Book) as []Book
value in return statement
```

Makes sense, doesn't it? The function's signature says it returns a `[]Book`, and that's what we still want it to do. But `catalog` is *not* of that type; it's of type `map[string]Book` instead. How can we turn one into the other?

Using `maps.Values`

Well, we *could* do it the laborious way. We could write a `for ... range` loop over the map, appending each book to a slice of results that we then return. That would work, but there's a more concise way to do it.

First, instead of ranging over the map to visit each book in turn, we can use the standard library `maps.Values` function to get all the elements of the map at once. To do that, we'll import the `maps` package:

```
import "maps"
```

Now we can write:

```
books := maps.Values(catalog)
```

Iterators

This is neat, but we're not quite done yet. What `maps.Values` gives us is, not a slice of `Book`, but an *iterator* of `Book`. You could think of an iterator as being like a "lazy slice": it doesn't actually produce any elements until you ask it to.

Since we very often want to visit each element in a sequence in turn, an iterator is perfect for this. It doesn't waste time computing all the elements before we need them, and it doesn't allocate the memory required to hold all the elements at once. It just yields one element at a time, and only when we actually ask for it.

To do that, we could use a `range` loop to visit each element in turn:

```
for book := range maps.Values(catalog) {
    ... // do something with each `book`
}
```

Using `slices.Collect`

But another very common thing to do with an iterator is to collect all its elements into a slice, and that's what we want to do here. We can use `slices.Collect` to do that. First, we'll import the `slices` package:

```
import "slices"
```

Then we can write `GetAllBooks` like this:

```
func GetAllBooks() []Book {
    return slices.Collect(maps.Values(catalog))
}
```

(Listing books/13)

To get a map's elements as a slice, then, we first of all get an iterator of the elements using `maps.Values`. Then we use `slices.Collect` to consume that iterator and collect all the elements into a slice. And that's our result.

Slices and maps have a lot in common

So, what changes do we need to make to `GetBook`? Here's what we have right now:

```
func GetBook(ID string) (Book, bool) {
    for _, book := range catalog {
        if book.ID == ID {
            return book, true
        }
    }
    return Book{}, false
}
```

(Listing books/12)

Surprisingly, perhaps, this still compiles, and still passes the test! That's because slices and maps are really quite similar: both work with `range`, for example, and both pro-

duce the results you'd expect. That makes sense if you think of a slice as a special case of a map, where the keys are only allowed to be integers.

With a slice, `range` gives us each index and element pair in turn. With a map, `range` gives us each *key* and element pair in turn.

That's what we're getting here, so each time round the loop, `book` is the next book in the map, and we check its ID against the one we're looking for. So this code works exactly as it did before, except that it's looping over a map rather than a slice.

Direct lookups in maps

That's not terrible, but recall that we were motivated to make this change in the first place by the fact that we don't want to have to search through $N/2$ books every time. We wanted a catalog type that lets us retrieve a book *directly* by ID, in a single $O(1)$ lookup, without searching.

And we have that now, so let's take advantage of it. How do we look up an element in a map by key? We can use exactly the same syntax as for looking up a slice element by index:

```
book := catalog[ID]
```

Not so fast, though: we know that the book with that ID might not *be* in the catalog. What happens then? What value gets assigned to `book`?

Zero values for structs

Well, it's the *zero value*, which you'll remember is whatever the default value is for that particular type. With `string`, for example, it's the empty string, and with `int`, it's `0`.

What about with a struct, then? For example, what would the zero value of a `Book` struct be? The answer is that it's a `Book` where every *field* has its default value. That is:

```
Book{
    Title:  "",
    Author: "",
    Copies: 0,
    ID:     "",
}
```

Okay, good to know, but as you'll recall from the previous chapter, this isn't how `GetBook` works. Instead, `GetBook` returns a second, `bool` value along with the `Book` result, indicating whether or not the book was found.

The ok syntax

So how do we know whether or not the book *was* found, if the map lookup returns only a "zero" `Book`? Well, it turns out there's a very convenient way to do that. We can receive *two* values from the map instead of one:

```
book, ok := catalog[ID]
```

And it's not a coincidence that we named the second value `ok`, because that's what it is: it's a `bool` value that's `true` if the book was found, or `false` otherwise. Just what we need!

Putting it all together

So here's the updated implementation of `GetBook`:

```
func GetBook(ID string) (Book, bool) {
    book, ok := catalog[ID]
    return book, ok
}
```

(Listing `books/13`)

This is nicer than the loop we had before, and its performance also scales a lot better. No matter how many books we have in stock, `GetBook` will always take more or less exactly the same time. Great!

Road testing

So, does our `find` program still work? Let's try:

```
go run ./cmd/find abc
```

```
In the Company of Cheerful Ladies by Alexander McCall Smith (copies: 1)
```

Nice. Let's check `list`, too:

```
go run ./cmd/list
```

```
In the Company of Cheerful Ladies by Alexander McCall Smith
(copies: 1)
White Heat by Dominic Sandbrook (copies: 2)
```

And one more time, just for luck:

```
go run ./cmd/list
```

```
White Heat by Dominic Sandbrook (copies: 2)
In the Company of Cheerful Ladies by Alexander McCall Smith
```

```
(copies: 1)
```

Wait, what? It looks like they came out in a different order the second time. Why would they do that?

Iteration order of maps

Well, as you probably suspected, the reason is to do with the fact that we're now storing the catalog in a map instead of a slice. A slice is, by definition, an *ordered* sequence, so when we loop over it, we always get the elements in index order.

That's not true of a map. While there's an obvious and sensible way to order a bunch of numbers, the keys of a map don't have to be numbers, or even strings. So, since there's no obvious order for the keys, when we iterate over a map like this we'll get its elements in *arbitrary* order, and it will be different from one run of the program to the next.

That's fine, and it doesn't matter in what order we print the books for the `list` program, but what about the `GetAllBooks` test? Doesn't that compare the result of `GetAllBooks` with a slice where the books are in a fixed order?

Flaky tests

Yes, it does, and that means we can't rely on this test always passing. Sometimes it will pass, just by coincidence, but there must be times when it will fail. And we can prove that, by running the test a bunch of times:

```
go test -count=100
```

The result of `GetAllBooks` is in the expected order about 90% of the time, but we don't want our tests failing for no good reason once in every ten runs. After all, the *program* is correct: it's just the test that's unreliable, because it's assuming something about the result of `GetAllBooks` that isn't always true.

And we *don't* care about the order of the books returned by `GetAllBooks`, only that they're all there. So we can fix this *flaky* test by sorting the result slice before we compare it to the expected one.

Sorting structs by field

So, how to do that? If we had a slice of some *ordered* type (that is, a type like `int` or `string` where the ordering of values is obvious), we could sort it easily by using `slices.Sort`. But that's not the case here.

`slices.Sort` won't work for a slice of `Book` because Go doesn't know how to compare two `Book` values. Should they be ordered by title, author, number of copies, or their ID fields? It's up to us to decide, and it doesn't matter for our purposes, so let's sort them by author.

To do that, we can use `slices.SortFunc`, which will sort a slice by asking *us* to provide a function that compares any two values. Here's what that looks like:

```
got := books.GetAllBooks()
slices.SortFunc(got, func(a, b books.Book) int {
    return cmp.Compare(a.Author, b.Author)
})
```

(Listing books/13)

The first argument to `SortFunc` is the slice to sort, reasonably enough. The second is the comparison function we're providing. It takes two `Book` values, which we call `a` and `b`, and returns a number indicating whether `a` should be sorted before or after `b`.

We don't need to do anything fancy here, since we just want to sort the books by author, so we use the `cmp.Compare` function to compare the `Author` fields of the two values. With this done, the test has now stopped flaking:

```
go test -count=100
```

```
PASS
```

Any time we're testing code that queries a map, like `GetAllBooks`, we'll probably want to use this technique to make sure the test is reliable. Since sorting is a relatively expensive operation, we don't want to do it in `GetAllBooks`, because users don't care about the order of the books; it only matters for testing.

Modifying the catalog

Great. So we've updated our bookstore code to store the catalog as a map, and we can list all books, or find a specific book efficiently (if it's in stock). This is all valuable stuff for Happy Fun Books operations, but there's one problem we haven't yet solved. How do books get into the catalog in the first place?

In other words, suppose a delivery of books arrives at one of our stores, and the staff now need to enter the details of the new titles into the catalog. How do they do that? Right now, the books in the catalog are fixed by the Go source code of the `books` package, so to add new books, we'd have to edit that source code and recompile the program. Not ideal!

The magic function

We'd like to be able to add (and, indeed, remove) books without changing the program itself. After all, the idea is that Happy Fun Books can scale up to many locations, each of which would have a different catalog of books, but all using the *same* store management software.

Let's design this new feature using what I sometimes call the "magic function" approach. That is, we imagine that there's some wonderful function that does exactly

what we want, in just the way we want it to. What would it look like to call that function?

The most straightforward way we could imagine adding a new book to the catalog is something like this:

```
AddBook(Book{
    ID:     "123",
    Title:  "The Prize of all the Oceans",
    Author: "Glyn Williams",
    Copies: 2,
})
```

That is, we're imagining that there's some function we can call, named `AddBook`, and we pass it a `Book` struct, and it adds that book to the catalog. Seems reasonable, doesn't it? Sure, this function doesn't exist yet, but we can already see that it would fit well into our existing package.

Testing `AddBook`

The name is obvious and says exactly what it does: `AddBook`. It makes sense that it would take a `Book` as its argument, and at the moment we can't think of anything that it would usefully return. So this seems like a good starting point.

GOAL: Write a new test for our (still imaginary) `AddBook` function. I'll leave the details up to you. Make sure it fails with a dummy implementation of `AddBook` that does nothing.

HINT: Now you're stretching your legs a bit as a Go developer. It's not just a matter of writing the code for the test: you've also got to decide what the test should *do*.

You've had some practice at this already, but if nothing occurs to you right away, try the trick of describing to yourself *in English* what a useful test would assert about the results of calling `AddBook`.

For example, when you call `AddBook`, *something* should change, but what? And how can you tell? See if you can figure it out.

SOLUTION: Here's my first version:

```
func TestAddBook_AddsGivenBookToCatalog(t *testing.T) {
    t.Parallel()
    books.AddBook(books.Book{
        ID:     "123",
        Title:  "The Prize of all the Oceans",
        Author: "Glyn Williams",
        Copies: 2,
    })
    _, ok := books.GetBook("123")
    if !ok {
        t.Fatal("added book not found")
    }
}
```

Reasonable, isn't it? We just add a certain book to the catalog, and then we check to see if it *was* added—if so, we should be able to retrieve it by calling `GetBook` with its ID ("123").

And this test does indeed fail with the dummy `AddBook`, so is that it? Are we ready to move on and implement the magic function, or do we need to mull over the test a little more? Have one more look at the code and see if you can spot any potential problems.

Postconditions and preconditions

Here's one interesting question we could ask about this test: what if, at some time in the future, a book with the ID "123" *already* exists in the catalog? Well, that would render the test completely useless!

To understand why, think about what the test would do under these circumstances if there *were* indeed a bug in `AddBook`. For example, suppose it did nothing at all, just as our current dummy implementation does. Would this test detect that bug?

No, it wouldn't. It would successfully retrieve a book with the expected ID from the catalog, even though it wasn't put there by `AddBook`. The test passes even if `AddBook` does nothing whatsover, and that's actually slightly worse than not having a test at all, because it still *looks* like we have a test.

What did we (okay, *I*) do wrong? I made a very common mistake in software testing, which is to be concerned only with the state of the world *after* calling the function under test.

In the jargon, these are called *postconditions*: the test asserts that the postcondition of `AddBook` is that the given book exists in the catalog. But it says nothing about the *preconditions*. It would be useful to assert that the same book *doesn't* exist *before* `AddBook` is called, wouldn't it?

A better AddBook test

Fine. Let's make that change:

```
func TestAddBook_AddsGivenBookToCatalog(t *testing.T) {
    t.Parallel()
    _, ok := books.GetBook("123")
    if ok {
        t.Fatal("book already present")
    }
    books.AddBook(books.Book{
        ID:     "123",
        Title:  "The Prize of all the Oceans",
        Author: "Glyn Williams",
        Copies: 2,
    })
    _, ok = books.GetBook("123")
    if !ok {
        t.Fatal("added book not found")
    }
}
```

Now we have a much more resilient test. Instead of merely asserting that the world is in a certain state after `AddBook` is called, the test asserts that calling `AddBook` actually *changes* the world. Isn't that what we all want to do as software engineers? Well, now we have a test for it.

A hint of disquiet

How should we implement `AddBook` for real, then? Well, first we need to know how to add a new element to a map. Here's what that looks like:

```
func AddBook(book Book) {
    catalog[book.ID] = book
}
```

Pleasingly straightforward, I think you'll agree. And this passes the test. So, are we done *now*?

Well, I don't know about you, but all this talk about preconditions and postconditions has made me a little nervous. We have several tests now, and they all make various assumptions about the state of the catalog (which, remember, is a package-level variable,

so it's shared by all the tests).

That was more or less fine as long as the catalog never changed, but now we *are* changing the catalog, by adding a new book to it. And we're using `t.Parallel()` in all our tests, to make them run concurrently. What that means in practice is that the tests won't run in a fixed order, one after another: each test will be executed independently, without regard to what other tests are doing.

That sounds like a recipe for problems, given that one test is modifying the catalog, and other tests are making assumptions about exactly what books are in the catalog. In fact, it seems like some of those other tests might fail sometimes, depending on whether or not the `AddBook` test has run before they look at the catalog.

Race conditions in parallel tests

So, did we just get lucky with the previous test run? Let's try the `-count=100` trick again:

```
go test -count=100
```

Oh dear:

```
--- FAIL: TestGetAllBooks_ReturnsAllBooks (0.00s)
    books_test.go:31: want []books.Book{books.Book{Title:"In the
    Company of Cheerful Ladies", Author:"Alexander McCall Smith",
    Copies:1, ID:"abc"}, books.Book{Title:"White Heat",
    Author:"Dominic Sandbrook", Copies:2, ID:"xyz"}}, got
    []books.Book{books.Book{Title:"In the Company of Cheerful
    Ladies", Author:"Alexander McCall Smith", Copies:1, ID:"abc"},
    books.Book{Title:"White Heat", Author:"Dominic Sandbrook",
    Copies:2, ID:"xyz"}, books.Book{Title:"The Prize of all the
    Oceans", Author:"Glyn Williams", Copies:2, ID:"123"}}
```

This is exactly what we feared: the `GetAllBooks` test is failing, even though `GetAllBooks` itself actually works perfectly. That's because (to spare your eyesight a little) the test is expecting the catalog to contain exactly and only these books:

```
In the Company of Cheerful Ladies
White Heat
```

And what it actually found was these:

```
In the Company of Cheerful Ladies
White Heat
The Prize of all the Oceans
```

You can see what's happened, can't you? On this particular test run, the `AddBook` test added the new book right before the `GetAllBooks` test requested the catalog, and as a result it found an unexpected book, which caused it to fail.

Stepping on each other's toes

And it gets worse, because not only are our existing tests failing, the *new* test is failing too:

```
--- FAIL: TestAddBook_AddsGivenBookToCatalog (0.00s)
    books_test.go:64: book already present
```

What's *that* about? Well, `count=100` doesn't just run the whole test suite a hundred times. It runs *each test* a hundred times, so since we have four tests, we have four hundred concurrent tests running—including a hundred instances of the `AddBook` test.

So suppose instance 1 of the test calls `AddBook`, and some time after that, instance 2 calls `GetAllBooks` to check the precondition that the book isn't in the catalog. But it is, because it was already added by the other instance.

This seems like a pretty nasty problem. We don't really want to give up running our tests in parallel, because it's a big time-saver. And we certainly don't want to give up being able to modify the catalog: that's the whole point of what we were trying to do!

But, on the other hand, we don't want flaky tests, because that undermines the whole point of *having* tests. What can we do? You might like to think about it a little before reading on to hear what I think.

Data races

Your first instinct might be to just take out the troublesome `t.Parallel()`, and I completely understand why. If running the tests in parallel makes them unreliable, surely the obvious answer is "don't do that".

But sadly, even this doesn't solve the problem. After all, it's not really a *test* problem. Suppose we have a large branch of Happy Fun Books, with several point-of-sale terminals. And suppose an associate is entering a new book into the catalog at terminal A, while another is simultaneously looking up the same book at terminal B. Does the lookup succeed or not?

"It depends," is the only answer we can give. If the call to `AddBook` happens one microsecond before the call to `GetBook`, then yes, the book will have been added, so it'll be found. But if `AddBook` happens one microsecond *after* `GetBook`, then of course the search won't find anything.

The two concurrent function calls are in a race, literally: whichever one arrives at the *critical section* first "wins", in some sense. That's why this kind of problem is called a *data race*: it happens when the result of some concurrent operations depends on the microscopic details of ordering and timing. And that's why our tests have become unstable.

De-globalising the catalog

The fact is that we *want* to be able to concurrently access the catalog, and not just for testing: real bookstores will want to do this, too. To make our tests reliable again, we need to eliminate the data race. So, how can we do that?

The simplest and safest way to fix the *test* problem is to prevent concurrent access to the catalog by giving each test its *own* catalog. That way, any individual test doesn't need to worry about the contents of the catalog changing underneath it, because no other test *can* change it.

Yes, in the real bookstore the concurrent terminals will still need to share a global catalog, and there are ways of making that safe, too, but let's tackle one problem at a time. First, what changes do we need to make to the existing code to eliminate the data race on `catalog`?

Making `catalog` a parameter

The first thing we'll need to do is to modify the functions in `books` so that, instead of referring to a single global `catalog` variable, they *take* the catalog as a parameter:

```
func GetAllBooks(catalog map[string]Book) []Book
func GetBook(catalog map[string]Book, ID string) (Book, bool)
func AddBook(catalog map[string]Book, book Book) {
```

And, happily, that's the only change we need to make to the `books` package, apart from removing the now-redundant `catalog` variable.

A test data helper

Let's now fix the tests, because each test will now need to create its own `catalog` and pass it to all the functions it calls. We could write something like this, for example:

```
catalog := map[string]books.Book { ... }
_, ok := books.GetBook(catalog, "123")
```

But it's going to be tedious and repetitive to keep defining the same map literal in every test. Let's move that out to a helper function:

```
catalog := getTestCatalog()
```

This is a useful pattern any time you want to use the same test data in each of a bunch of tests: abstract that setup into a little function. It's easy to write:

```
func getTestCatalog() map[string]books.Book {
    return map[string]books.Book{
        "abc": {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
            ID:     "abc",
        },
        "xyz": {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
            ID:     "xyz",
        },
    }
}
```

(Listing books/14)

And now every test can call `getTestCatalog` to get a crisp, new catalog of its own, without worrying about accidentally stomping on someone else's copy.

Fixing the tests

Finally, we just need to modify all the calls to `books` functions in the tests, to add `catalog` as the first parameter:

```
got := books.GetAllBooks(catalog)
```

...and so on. Here's our updated test for `AddBook`, for example:

```
func TestAddBook_AddsGivenBookToCatalog(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    _, ok := books.GetBook(catalog, "123")
    if ok {
        t.Fatal("book already present")
    }
    books.AddBook(catalog, books.Book{
```

```
        ID:     "123",
        Title:  "The Prize of all the Oceans",
        Author: "Glyn Williams",
        Copies: 2,
    })
    _, ok = books.GetBook(catalog, "123")
    if !ok {
        t.Fatal("added book not found")
    }
}
```

(Listing books/14)

With this change complete, we can now run the tests as many times as we like in parallel and they'll pass reliably. No more data races!

Initializing the real catalog

There's one thing left to fix. Our `find` and `list` commands don't actually work anymore, since we removed the `catalog` variable from `books`. But we need *some* real catalog for these programs to operate on, so where will that data come from?

In other words, in the `find` command, for instance, we're going to write something like this, passing `catalog` as an argument to `GetBook`:

```
book, ok := books.GetBook(catalog, ID)
```

That means we need to assign something to `catalog`. In the tests we solved this problem by calling a function that returns the catalog as a map, so let's use the same idea here:

```
catalog := books.GetCatalog()
```

And, for now, we can have that function return the exact same map we're using in the tests. So here's our updated version of `list`, for example:

```
func main() {
    catalog := books.GetCatalog()
    for _, book := range books.GetAllBooks(catalog) {
        fmt.Println(books.BookToString(book))
    }
```

```
}
```

(Listing books/14)

With this seemingly rather superficial change, we've added a lot of value to the `books` package. Not only can we run any number of tests reliably in parallel, even if they modify the contents of the catalog, we've also made it so that individual bookstores can now have *different* catalogs, in principle.

In the next chapter, we'll extend this idea a little further, turning both books and catalogs into *objects* that encapsulate both data and behaviour.

Here's what we know so far

You've now mastered maps as a data structure, and that's huge: practically every program you ever write in Go will use maps in one way or another. You know how to declare, query, modify, and pass around maps as data from one function to another.

You also learned something about the scaling performance of different algorithms, such as linear search versus direct access, and the big-O notation we use to talk about them: $O(N)$ versus $O(1)$, for example. That means you understand when it makes sense to store data in a map instead of a slice, which is useful.

And, importantly, you're starting to get the hang of concurrent programming in Go, since you now know what data races are, why they're a problem, and at least one way to avoid them. Great work! See you in the next chapter.

6. Objects behaving badly

In the previous chapter we did some useful work building a more efficient data structure to store our books, turning `catalog` into a map of books by ID. That's great, but our code is starting to look a bit cluttered up with paperwork.

For example, every time we call a function involving the catalog, we have to pass `catalog` as one of the arguments:

```
catalog := books.GetCatalog()
books.AddBook(catalog, newBook)
book, ok := books.GetBook(catalog, newBook.ID)
for _, book := range books.GetAllBooks(catalog) {
    // and so on
}
```

Objects and methods

Fortunately, Go provides a way to express the same idea more concisely, using *objects* and *methods*. And, as complicated and unfamiliar as that might sound, we've actually been using objects and methods all along, without necessarily being aware of it.

The mysterious `t`

For example, consider this test:

```
func TestGetBook_ReturnsFalseWhenBookNotFound(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    _, ok := books.GetBook(catalog, "nonexistent ID")
    if ok {
        t.Fatal("want false for nonexistent ID, got true")
    }
}
```

(Listing books/14)

And let's take a closer look at what's happening with this mysterious parameter `t`. First, we do something with it that makes this test run in parallel:

```
t.Parallel()
```

We've seen function calls before where the name of the function is prefixed by some package: for example, `fmt.Println` is in the `fmt` package. That's not what *this* is, though; there's no package called `t`.

Methods on `t`

Instead, `t` is a parameter, so it contains some Go value—some data, in other words. When we use `t` as a prefix to a function name like `Parallel`, we're calling the function *on* that value: that is, whatever `t` contains.

`Parallel`, in fact, is not an ordinary function like those we've written before. It can *only* be called on a value of type `*testing.T`, so we say that `Parallel` is a *method* of this type.

And later on, when we want the test to fail, we call another method on `t`:

```
t.Fatal("want false for nonexistent ID, got true")
```

You might wonder why the `t` is needed at all. Why aren't these methods simply functions in the `testing` package? Couldn't we call instead, for example:

```
testing.Fatal("oh no")
```

That would tell Go that *some* test has failed, but not which one, and that's important. We have lots of tests, and we want to control their failure (and maybe parallelism) individually, so we need *some* value that uniquely identifies this particular test.

Methods are syntactic sugar for functions

Since the `t` parameter already plays this role, we could imagine passing it to the function as an argument, like this:

```
testing.Fatal(t, "oh no")
```

And, actually, that's exactly what we do, except we can use a more concise syntax:

```
t.Fatal("oh no")
```

You can see what's happening, can't you? The method notation (`t.Fatal()`) is simply what's called *syntactic sugar*: a sweet shorthand for passing the `t` as the first argument to the function. Under the hood, Go transforms our method call into a plain old function where the `t` is the first argument.

Defining methods

So how does this work, exactly? Can we turn any function call into a method call? For example, could we turn `fmt.Println("hello")` into:

```
"hello".Println()
```

No, that's an error:

```
"hello".Println undefined (type untyped string has no field or
method Println)
```

Instead, we have to define functions in a special way to turn them into methods. Let's see how to do this with our `BookToString` function, for example.

Turning `BookToString` into a method

We're saying that instead of passing some `Book` value to the function, we'd like to be able to call the function as a method *on* some `Book`. Instead of:

```
fmt.Println(books.BookToString(book))
```

we'd like to be able to write:

```
fmt.Println(book.BookToString())
```

In fact, let's shorten it to just `String`, to avoid repeating ourselves:

```
fmt.Println(book.String())
```

That looks nice, but how do we change the function definition to make this possible? Here's what we have right now:

```
func BookToString(book Book) string {
    return fmt.Sprintf("%v by %v (copies: %v)",
        book.Title, book.Author, book.Copies)
}
```

(Listing `books/14`)

The receiver

Turning this into a method means promoting the `book` parameter to what's called the method's *receiver*: not just any old parameter, in other words, but the most important parameter of all.

```
func (book Book) String() string {
    return fmt.Sprintf("%v by %v (copies: %v)",
        book.Title, book.Author, book.Copies)
}
```

(Listing `books/15`)

All we've done is move the `book` parameter from after the function name to before it, but that makes all the difference. It tells Go that `String` must be called on some `Book` value, and inside the body of the method, that value is available as `book`.

Now we're able to write:

```
fmt.Println(book.String())
```

Calling `String` automatically

That's nice, but in this particular case we can do even better. It's so common to need to *stringify* values that `fmt.Println` will automatically call a method named `String` if one exists. So, now that we've provided such a method, we can simply pass the `book` value directly:

```
fmt.Println(book)
```

The `fmt` package can always figure out a sensible way to print something, but if we've defined a `String` method for the type, that will be used instead.

Updating the tools

Now we can tidy up our `list` and `find` tools to use the new, more concise syntax. For example, here's `list`:

```
func main() {
    catalog := books.GetCatalog()
    for _, book := range books.GetAllBooks(catalog) {
        fmt.Println(book)
    }
}
```

Adding methods to the catalog

Very neat. So, could we use the same trick with the functions that take `catalog`, and turn them into methods too? Here's our current version of `GetAllBooks`, for example:

```
func GetAllBooks(catalog map[string]Book) []Book {
    return slices.Collect(maps.Values(catalog))
}
```

(Listing `books/14`)

Methods can only be added to local types

We know the rule for turning a function into a method: move the `catalog` parameter before the function name.

So, can we do this, for example?

```
func (catalog map[string]Book) GetAllBooks() []Book {
    return slices.Collect(maps.Values(catalog))
}
```

It looks okay at first sight, but Go isn't happy about it:

```
invalid receiver type map[string]Book
```

It turns out there's one more thing we need to know about methods: we can only define them on our own, "local" types—that is, types we've defined in the same package.

We can't add a method to a built-in type like `string` for the same reason:

```
func (s string) Print() {
    fmt.Println(s)
}
// cannot define new methods on non-local type string
```

Defining a new type

In a sense, the methods available on a given type are *part* of that type's definition. So we can't change someone else's type by adding our own methods to it: that would be like building an extension on your neighbour's house without their permission.

But there's an easy way around this: we can just define a new type, and then we're allowed to add methods to it. When the house is our own, we can build any extensions we want.

Let's go ahead and create a new type `Catalog`, then:

```
type Catalog map[string]Book
```

(Listing books/15)

Now we can rewrite `GetAllBooks` as a method with no problems:

```
func (catalog Catalog) GetAllBooks() []Book {
    return slices.Collect(maps.Values(catalog))
}
```

(Listing books/15)

`Catalog` is a distinct type

Note that we have to use the type name `Catalog` for the receiver, though. We can't say instead `map[string]Book` here, because `Catalog` isn't just a new *name* for that type, it's a whole new type of its own.

The `GetBook` and `AddBook` methods

GOAL: Go ahead and make the necessary changes to turn `GetBook` and `AddBook` into methods on `Catalog` too, and fix the tests.

HINT: You know how to change the function signatures, in the same way as we did for `GetAllBooks`, and you'll also need to change all references to `map[string]Book` to `Catalog` to make this work. Don't forget to update the tests, and the code for the `list` and `find` tools, to use the new method calls instead of function calls.

SOLUTION: Here's the updated version of `GetBook`:

```
func (catalog Catalog) GetBook(ID string) (Book, bool) {
    book, ok := catalog[ID]
    return book, ok
}
```

(Listing books/15)

Here's `AddBook`:

```
func (catalog Catalog) AddBook(book Book) {
    catalog[book.ID] = book
}
```

(Listing books/15)

Updating the tests to call methods

Now we need to update the tests accordingly:

```
func TestGetAllBooks_ReturnsAllBooks(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    want := []books.Book{
        {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
            ID:     "abc",
        },
        {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
            ID:     "xyz",
        },
    }
    got := catalog.GetAllBooks()
    slices.SortFunc(got, func(a, b books.Book) int {
        return cmp.Compare(a.Author, b.Author)
    })
    if !slices.Equal(want, got) {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}

func TestGetBook_FindsBookInCatalogByID(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    want := books.Book{
        ID:     "abc",
        Title:  "In the Company of Cheerful Ladies",
        Author: "Alexander McCall Smith",
        Copies: 1,
    }
    got, ok := catalog.GetBook("abc")
    if !ok {
```

```
        t.Fatal("book not found")
    }
    if want != got {
        t.Fatalf("want %#v, got %#v", want, got)
    }
}

func TestGetBook_ReturnsFalseWhenBookNotFound(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    _, ok := catalog.GetBook("nonexistent ID")
    if ok {
        t.Fatal("want false for nonexistent ID, got true")
    }
}

func TestAddBook_AddsGivenBookToCatalog(t *testing.T) {
    t.Parallel()
    catalog := getTestCatalog()
    _, ok := catalog.GetBook("123")
    if ok {
        t.Fatal("book already present")
    }
    catalog.AddBook(books.Book{
        ID:     "123",
        Title:  "The Prize of all the Oceans",
        Author: "Glyn Williams",
        Copies: 2,
    })
    _, ok = catalog.GetBook("123")
    if !ok {
        t.Fatal("added book not found")
    }
}
```

(Listing books/15)

The test catalog

This in turn means a change to `getTestCatalog`:

```
func getTestCatalog() books.Catalog {
    return books.Catalog{
        "abc": {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
            ID:     "abc",
        },
        "xyz": {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
            ID:     "xyz",
        },
    }
}
```

(Listing books/15)

Updating the tools

Then, we'll update the `main` function for the `list` tool:

```
func main() {
    catalog := books.GetCatalog()
    for _, book := range catalog.GetAllBooks() {
        fmt.Println(book)
    }
}
```

(Listing books/15)

and for `find`:

```
func main() {
    if len(os.Args) != 2 {
        fmt.Println("Usage: find <BOOK ID>")
```

```
        return
    }
    catalog := books.GetCatalog()
    ID := os.Args[1]
    book, ok := catalog.GetBook(ID)
    if !ok {
        fmt.Println("Sorry, I couldn't find that book in the catalog.")
        return
    }
    fmt.Println(book)
}
```

(Listing books/15)

Updating `GetCatalog`

Finally, we need to change `GetCatalog` to return the new type:

```
func GetCatalog() Catalog {
    return Catalog{
        "abc": {
            Title:  "In the Company of Cheerful Ladies",
            Author: "Alexander McCall Smith",
            Copies: 1,
            ID:     "abc",
        },
        "xyz": {
            Title:  "White Heat",
            Author: "Dominic Sandbrook",
            Copies: 2,
            ID:     "xyz",
        },
    }
}
```

(Listing books/15)

Pointers

Although there are quite a few small changes here, they're pretty superficial: as we've seen, methods are really just syntactic sugar for functions where one of the parameters is given a special importance.

Objects as abstractions

In another way, though, the changes we've made are fundamental, because now the `Book` and `Catalog` types are not just a way of storing data. Adding methods to them gives them *behaviour*: they can do things. A `Catalog` is not just a passive bunch of books: it also knows how to add and retrieve books from itself, and how to list all the books it contains.

Values like this, that encapsulate both data and behaviour, are sometimes known as *objects*, though that's not a term that has any special meaning in Go. It's just a way of thinking about programs, or, more specifically, abstractions.

We saw in a previous chapter that abstractions are useful for making programs easier to write and reason about, because they hide irrelevant details behind a simple interface. A function is the simplest kind of abstraction, but an object is a more sophisticated one.

You could think of objects as "smart data": data that knows how to manipulate itself in useful ways. Alternatively, you could regard an object as "stateful code": not just a collection of related functions plus static data, but a unique *entity* that can change and develop over its "lifetime".

Validation

One very useful thing that objects can do to be smart about their data is to *validate* it. For example, it might already have struck you that it's possible to set the `Copies` field of a book to a negative number:

```
book.Copies = -1 // apparently this is fine
```

Go's `int` type represents whole numbers in general, and whole numbers can be negative, so this is allowed. But it doesn't really make sense for a number representing *copies of a book* to be negative. What would that even mean? We're describing a number of physical things, and there can't be less than no copies of a book.

Of course, we could add checks throughout the bookstore code to make sure that every time we deal with some `Book` value, it has a non-negative number of copies. But it would be annoying and wasteful to repeat that checking code in many places, and we might also want to validate other fields, not just `Copies`.

Also, we'd like to be able to publish the `books` package as an open-source software component that other people can import and use in their own Go programs that deal with

book information. It would be a shame if we put a similar burden of constant checking and re-checking on those users, and if they forgot to do it in even one place, their programs could produce incorrect results.

Making `Copies` idiot-proof

The fundamental problem here is that it's possible to create a `Book` value that is *invalid*, whatever we decide "invalid" means in this context. Right now, one way that we've identified for books to be invalid is for them to have a negative number of copies.

Ideally, we'd eliminate this problem by making it impossible for users (including us, as we're working on the bookstore code) to set `Copies` to a negative number.

But how can we prevent people from setting the `Copies` field to any `int` value that they like? There doesn't seem to be a way to do that in Go.

And there isn't, so we'll have to be a bit more creative. What if, instead of setting the `Copies` field directly, by using an assignment statement, we provided a *method* that would set `Copies` for us? In other words, what if we could write code like this to update the book:

```
book.SetCopies(12)
```

If we had a `SetCopies` method that took some `int` parameter, then we could do the validation—checking that it's not negative—in one place, once and for all, and it would be used automatically every time someone updated the number of copies of a book.

A non-validating `SetCopies` method

Before we start thinking about the validation, let's try to write the simple-minded version of `SetCopies`: one that just updates the `Copies` field to whatever we pass it, without trying to check whether it's negative. We'll start with a test:

```
func TestSetCopies_SetsNumberOfCopiesToGivenValue(t *testing.T) {
    t.Parallel()
    book := books.Book{
        Copies: 5,
    }
    book.SetCopies(12)
    if book.Copies != 12 {
        t.Errorf("want 12 copies, got %d", book.Copies)
    }
}
```

This doesn't seem unreasonable, does it? We create a new `Book` value, not bothering to set any of the fields except `Copies`, because we don't care about them for this test.

We set the initial `Copies` to 5, and then we call `SetCopies(12)`. After this, we feel justified in asserting that `book.Copies` should be 12. If not, why not?

As usual, let's check the test by writing a stub version of `SetCopies` that does nothing:

```
func (book Book) SetCopies(copies int) {}
```

This should fail the test, we think, and indeed that's the case:

```
want 12 copies, got 5
```

An unexpected problem

So let's go ahead and have `SetCopies` actually update the number of copies:

```
func (book Book) SetCopies(copies int) {
    book.Copies = copies
}
```

Does *this* pass the test?

```
want 12 copies, got 5
```

Hmm. It seems we have a problem. The code looks correct, so let's add some print statements to help debug what's going on:

```
func (book Book) SetCopies(copies int) {
    fmt.Println("before update, book.Copies =", book.Copies)
    book.Copies = copies
    fmt.Println("after update, book.Copies =", book.Copies)
}
```

This is often a helpful way to figure out exactly why code isn't behaving as we expect. Here's the output when we run the test:

```
before update, book.Copies = 5
after update, book.Copies = 12
--- FAIL: TestSetCopies_SetsNumberOfCopiesToGivenValue (0.00s)
    books_test.go:89: want 12 copies, got 5
```

Even more puzzling! It seems that, after we set `book.Copies` inside the method, the value is indeed correct. Yet, when we return from `SetCopies`, the value of `Copies` appears to have changed back to its original value. What's going on?

Assignment creates a copy

To understand that, we need to know a little about how Go manages data in your computer's memory. In an earlier chapter, we talked about variables, and we said that a variable is a way of giving a name to a particular data value in your program:

```
x := 5
```

This creates the variable `x` and assigns the value `5` to it. So far, so good. But what if we now create another variable `y`, and assign `x` to it?

```
y := x
```

What value does `y` contain? That's easy, right? It's `5`.

```
fmt.Println(y)
// Output:
// 5
```

So now let's ask: what happens if we change the value of `x`? Does it also affect `y`?

```
x = 10
fmt.Println(y)
// Output:
// 5
```

No, it doesn't. Even though we've now set `x` to `10`, the value of `y` is still `5`, just as it was before.

So the assignment statement `y := x` doesn't create any kind of relationship between the variables `y` and `x`. It just copies whatever value is currently in `x` into `y`. Subsequent changes to `x` affect only `x`.

Function parameters are also passed "by copy"

The same thing happens when we pass a variable such as `x` to a function (or a method, which is a function that's associated with a particular type, like `Book`). The value of `x` gets copied, and the function receives a copy in its parameter. For example:

```
func display(value int) {
    fmt.Println(value) // `value` is a copy of whatever was passed
}
```

It follows, then, that if we *modify* that parameter inside the function, that change has no effect on the original variable that we got a copy of.

```
func display(value int) {
    value = 0 // only affects our local copy, not the original
}
```

The `SetCopies` bug

Now that we know this, it's clear that our `SetCopies` method suffers from the same problem:

```
func (book Book) SetCopies(copies int) {
    book.Copies = copies // only affects local `book`, not the original
}
```

In fact, if you're using an editor or IDE with Go language support, you'll get a warning about this line to that effect:

```
func (book Book) SetCopies(copies int) {
    book.Copies = copies // unusedwrite: unused write to field Copies
}
```

As usual, this is a bit cryptic if you don't know what's going on, but now we do. Methods whose receiver type is a value, not a pointer, are free to modify their receivers, but the changes have no lasting effect. When the method returns, the contents of the local variable `book` will just be lost.

But in this case, we *want* our change to affect the original book in the calling code, not just the temporary copy inside the method body. So how can we arrange that?

Pointers are references

Luckily, Go provides a way to do this, using a type called a *pointer*. You can think of a pointer value as a *reference* to some variable, rather than a copy of it. Here's an example:

```
x := 5
y := &x // `y` is a pointer to `x`
x = 10
fmt.Println(*y) // asking "what value does `y` point to?"
// Output:
```

```
// 10
```

Here, we're setting up some variable x whose value is 5, and then we're creating a *pointer* to that variable, with this line:

```
y := &x
```

The & and * operators

The & is Go's *address* operator. Instead of copying the value in x, it creates a *pointer* to x and stores it in the variable y. In effect, we've set up a permanent relationship between y and x. As the value of x changes over time, we'll see those changes reflected when we use the y pointer:

```
fmt.Println(*y)
```

Here's another new piece of syntax: `*y` *dereferences* y. That is, it retrieves the value that y "points" to (in this case, the contents of x).

Copies versus references: an analogy

This might sound very abstract and complicated, so let's think about a real-world analogy. Suppose I publish a fascinating blog post about Go, explaining all about pointers and values. And suppose you're inspired by this to write your own post (I hope you will be!)

You might be kind enough to want to quote my article, and you could do that by copying and pasting the text into your own piece. This is like a straightforward value assignment in Go: `y := x`.

Of course, if I edit and update my post in the future, which is quite likely, then your copy of it will be out of date. So, instead of copying it, you could use a web link instead:

`https://bitfieldconsulting.com/posts/pointers`

This is like using the address operator in Go to set up a link between two variables: `y := &x`.

When your readers follow the link, they'll see whatever the current contents of my post are. If I change it, and they follow the link again, they'll see those changes, too. This is like dereferencing a Go pointer:

```
fmt.Println(*y)
```

Pointers can be "write" as well as "read"

Sometimes we want a copy, so that we can make changes without affecting the original, and sometimes we want a pointer, so that we can see changes to the original. We can also use a pointer to *make* changes to the original. For example, if we change the value of `*y`, it stands to reason that `x` is updated, too:

```
*y = 20
fmt.Println(x)
// Output:
// 20
```

The relationship works both ways, in other words. And that's just what we want for our `SetCopies` method, isn't it? If it took as its receiver, not a `Book`, but a *pointer* to `Book`, then the change we make in the method *will* affect the original.

So we could write something like this:

```
func (book *Book) SetCopies(copies int) {
    (*book).Copies = copies
}
```

Notice that the receiver `book` has changed from a `Book` to a `*Book`.

First, we dereference the `book` pointer to get the original `Book` struct, with `(*book)`. Then we update its `Copies` field to the value we want.

Automatic dereferencing of struct pointers

This passes the test, and it's fine, but it's a little awkward that we have to use parentheses here:

```
(*book).Copies = copies
```

Fortunately, we use pointers to structs so often that Go provides an *automatic dereferencing* shorthand, to make this easier to write:

```
book.Copies = copies
```

Pointers can't have fields, so there's no ambiguity here: Go knows that if `book` is a pointer, then `book.Copies` *must* refer to a field on whatever struct value `book` points to.

Pointer types

So, what type is a pointer? That is to say, if we had to declare a variable that stores pointers, which we will, what type would we declare for it?

```
x := 5
var y *int
y = x
```

Here, the type of `y` is `*int` (pronounced "pointer to `int`"). This time, the `*` operator isn't dereferencing anything: it just means "pointer".

So there isn't a single "pointer" type: instead, the type of a pointer variable is "pointer *to* some other type" (for example, `int`). Or, in the case of our `SetCopies` method, "pointer to `Book`"; that is, `*Book`.

Adding validation to `SetCopies`

Sorry about that lengthy detour to introduce and explain pointers, but it was an important one. Without pointers, we can't write methods that modify their receiver. And without pointer methods (that is, methods whose receivers are pointers, as opposed to values), we can't write *validating* methods, which is what we set out to do.

When validation fails

And we could add some validation logic to `SetCopies` now, but there's a slight problem. What should happen if the provided `copies` value is invalid?

```
book.SetCopies(-1) // what should happen here?
```

We have some options. For one thing, we could have `SetCopies` *panic* if the validation fails. That would certainly prevent programs from creating invalid `Book` values, but we don't really want to stop the programs altogether. It would be better if `SetCopies` returned some value indicating whether or not the validation succeeded.

So what should we return? We could use `bool`, as we did with `GetBook` to indicate whether or not the requested book was found in the catalog. After all, there are two possible outcomes of calling `SetCopies`: either the value was valid or it wasn't. We could represent those as `true` and `false`, respectively.

That's not terrible, but if we just returned `false` when the value fails validation, we can't include any information about *why* it was invalid. Yes, in this context there's only one way that `copies` could be invalid: because it's negative. But we could imagine other kinds of validation, and indeed we'd like a solution for validating methods in general, one that would let us include a message explaining why validation failed.

Returning errors

Go has a special type for exactly this kind of situation: it's called `error`. We can use `error` values to indicate whether or not some operation went wrong, and if it did, *what* specifically went wrong. Here's an example:

```
err := book.SetCopies(-1)
if err != nil {
    fmt.Println("Oh dear, something went wrong:", err)
}
```

Here, we're calling `SetCopies` on some book with a sketchy input (`-1`), and we're receiving some return value which we assign to the variable `err`. That name doesn't have any special meaning to Go, but it is a very strong convention among Go programmers that error variables are always called `err`, so we'll follow it here.

Now, if the call to `SetCopies` succeeded—that is, there's no error to report—then the value of `err` will be the special value `nil`, meaning "no error".

On the other hand, if it's anything *other* than `nil`, then some error happened, and the value of `err` will presumably explain what the problem was. We can print that value using `fmt.Println`, for example.

Testing for "no error"

So, if we're going to have `SetCopies` return an `error` value, then we first of all need to update our existing test. This is a happy path test, remember, meaning that we call `SetCopies` with a valid number, so in this case the value of `err` should certainly be `nil`.

```
func TestSetCopies_SetsNumberOfCopiesToGivenValue(t *testing.T) {
    t.Parallel()
    book := books.Book{
        Copies: 5,
    }
    err := book.SetCopies(12)
    if err != nil {
        t.Fatal(err)
    }
    if book.Copies != 12 {
        t.Errorf("want 12 copies, got %d", book.Copies)
    }
}
```

(Listing books/16)

Notice that now we assign the result of `SetCopies` to the `err` variable. Then we check and fail the test if `err != nil`, because there *shouldn't* be an error with this particular test input. If there is, `SetCopies` must be incorrect, because it's returning an error for a valid input.

This is a more complex test than before, because we're saying that `SetCopies` now does *two* things if the input is valid: first, it returns a `nil` value for the error, and secondly, it updates `book.Copies`.

Let's update `SetCopies` to pass this test, then:

```
func (book *Book) SetCopies(copies int) error {
    book.Copies = copies
    return nil
}
```

Notice that now this method returns a result, of type `error`, and at the end of the method we explicitly return `nil`.

Testing for a validation error

This passes the test, but of course we don't actually have any validation yet, so we need a test for that, too. In the new test, we'll pass some *invalid* value to `SetCopies`, and we'll test that the result is *not* `nil`.

```
func TestSetCopies_ReturnsErrorIfCopiesNegative(t *testing.T) {
    t.Parallel()
    book := books.Book{}
    err := book.SetCopies(-1)
    if err == nil {
        t.Error("want error for negative copies, got nil")
    }
}
```

(Listing books/16)

We don't bother setting the `Copies` field to any initial value, since we won't be bothering to look at the *final* value: it shouldn't have changed. All we care about is that the `err` result is not `nil`. If it *is* `nil`, there's something terribly wrong with `SetCopies`, so we fail the test and report that fact.

Naturally enough, this test doesn't pass with the current version of `SetCopies`: how could it, since the method *always* returns `nil`, no matter what. We need to add some

code to detect invalid inputs, and return some error in that case.

Creating error values

So, how do we create error values when we need to return them? One way is to use the `errors.New` function, which as the name suggests creates a new error value:

```
return errors.New("invalid number of copies")
```

The value created by `errors.New` is not `nil`, so it would pass our sad-path test. If we printed that value with `fmt.Println`, it would print exactly the string that we put into it:

```
err := errors.New("invalid number of copies")
fmt.Println(err)
// Output:
// invalid number of copies
```

That's okay, but we can do a little more. We can also include *formatted* information in the error, using the `fmt.Errorf` function:

```
err := fmt.Errorf("negative number of copies: %d", -1)
fmt.Println(err)
// Output:
// negative number of copies: -1
```

This sounds like just the thing we want for `SetCopies`, so let's try it:

```
func (book *Book) SetCopies(copies int) error {
    if copies < 0 {
        return fmt.Errorf("negative number of copies: %d", copies)
    }
    book.Copies = copies
    return nil
}
```

(Listing books/16)

This passes both `SetCopies` tests, so it looks like we're in great shape!

Here's what we know so far

It's been a busy chapter, as you've now mastered two key concepts in Go: methods and pointers. You understand that methods are "syntactic sugar" for functions that take their first parameter as a *receiver*, meaning the thing that the method is about. For example, all the methods on `Catalog` take a catalog as their receiver.

You've connected this to your existing knowledge about the `testing` package, and you can now recognise calls such as `t.Fatal` as referring to methods on the `*testing.T` object: the thing that uniquely identifies each test and controls its execution.

You also learned the idea of a pointer, and why we might want to use one in a Go program. When a function takes some value parameter, it gets a transient copy of the data, and can't make lasting changes to it. When it takes a pointer parameter, though, it's getting a *reference* to the original data, and can use that reference to modify it permanently.

You know that we can use the `&` (address) operator to create a pointer to a value, and the `*` (dereference) operator to find the value that a pointer points to. And you also know that when this value is a struct, we don't need to bother with the `*`: we can refer to fields directly, and Go will take care of the dereferencing for us.

Nice job! In the next chapter, let's turn to the question of how to update book information in the catalog.

There's more to come!

Enjoying the book so far? This is the early access edition, so watch this space: more content is coming. In effect, you're reading this book as it's being written.

As I complete each new section of the book, I'll upload it to the Bitfield Consulting store (and I'll notify all the early access purchasers by email, too). And, of course, even after it's officially finished and released in full, I'll continue to make updates, and you'll get them for free.

See the instructions about "free updates to future editions" in the "About this book" section that follows, for more information about how to download your updates.

About this book

Who wrote this?

John Arundel is a Go teacher and mentor of many years experience. He's helped literally thousands of people to learn Go, with friendly, supportive, professional mentoring, and he can help you too. Find out more:

- Learn Go remotely with me

Feedback

If you enjoyed this book, let me know! Email go@bitfieldconsulting.com with your comments. If you didn't enjoy it, or found a problem, I'd like to hear that too. All your feedback will go to improving the book.

Also, please tell your friends, or post about the book on social media. I'm not a global mega-corporation, and I don't have a publisher or a marketing budget: I write and produce these books myself, at home, in my spare time. I'm not doing this for the money: I'm doing it so that I can help bring the love of Go to as many people as possible.

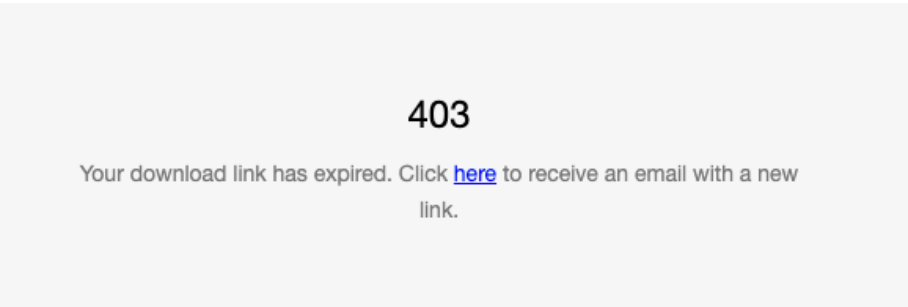
That's where you can help, too. If you love Go, tell a friend about this book!

Free updates to future editions

All my books come with free updates to future editions, for life. Make sure you save the email containing your download link that you got when you made your purchase. It's your key to downloading future updates.

When I publish a new edition, you'll get an email to let you know about it (make sure you're subscribed to my mailing list). When that happens, re-visit the link in your original download email, and you'll be able to download the new version.

NOTE: The Squarespace store makes this update process confusing for some readers, and I heartily sympathise with them. Your original download link is only valid for 24 hours, and if you re-visit it after that time, you'll see what *looks* like an error page:



But, in tiny print, you'll also see a "click here" link. Click it, and a new email with a new download link will be sent to you.

Join my Code Club

I have a mailing list where you can subscribe to my latest articles, book news, and special offers. I'd be honoured if you'd like to be a member. Needless to say, I'll never share your data with anyone, and you can unsubscribe at any time.

- Join Code Club

The Power of Go: Tools

Are you ready to unlock the power of Go, master obviousness-oriented programming, and learn the secrets of Zen mountaineering? Then you're ready for The Power of Go: Tools.

It's the next step on your software engineering journey, explaining how to write simple, powerful, idiomatic, and even beautiful programs in Go.

This friendly, supportive, yet challenging book will show you how master software engineers think, and guide you through the process of designing production-ready command-line tools in Go step by step.

The Power of Go: Tests

What does it mean to program with confidence? How do you build self-testing software? What even is a test, anyway? The Power of Go: Tests answers these questions, and many more.

Welcome to the thrilling world of fuzzy mutants and spies, guerilla testing, mocks and crocks, design smells, mirage tests, deep abstractions, exploding pointers, sentinels and six-year-old astronauts, coverage ratchets and golden files, singletons and walking skeletons, canaries and smelly suites, flaky tests and concurrent callbacks, fakes, CRUD methods, infinite defects, brittle tests, wibbly-wobby timey-wimey stuff, adapters and ambassadors, tests that fail only at midnight, and gremlins that steal from the player during the hours of darkness.

> *If you get fired as a result of applying the advice in this book, then that's probably for the best, all things considered. But if it happens, I'll make it my personal mission to get you a job with a better company: one where people are rewarded, not punished, for producing software that actually works.*

Go's built-in support for testing puts tests front and centre of any software project, from command-line tools to sophisticated backend servers and APIs. This accessible, amusing book will introduce you to all Go's testing facilities, show you how to use them to write tests for the trickiest things, and distils the collected wisdom of the Go community on best practices for testing Go programs. Crammed with hundreds of code examples, the book uses real tests and real problems to show you exactly what to do, step by step.

You'll learn how to use tests to design programs that solve user problems, how to build reliable codebases on solid foundations, and how tests can help you tackle horrible, bug-riddled legacy codebases and make them a nicer place to live. From choosing informative, behaviour-focused names for your tests to clever, powerful techniques for managing test dependencies like databases and concurrent servers, The Power of Go: Tests has everything you need to master the art of testing in Go.

Know Go

Things are changing! Go beyond the basics, and master the new generics features introduced in Go. Learn all about type parameters and constraints in Go and how to use them, with this easy-to-read but comprehensive guide.

If you're new to Go and generics, and wondering what all the fuss is about, this book is for you! If you have some experience with Go already, but want to learn about the new

generics features, this book is also for you. And if you've been waiting impatiently for Go to just get generics already so you can use it, don't worry: this book is for you too!

You don't need an advanced degree in computer science or tons of programming experience. Know Go explains what you need to know in plain, ordinary language, with simple examples that will show you what's new, how the language changes will affect you, and exactly how to use generics in your own programs and packages.

As you'd expect from the author of The Deeper Love of Go and The Power of Go: Tools, it's fun and easy reading, but it's also packed with powerful ideas, concepts, and techniques that you can use in real-world applications.

Further reading

You can find more of my books on Go here:

- Go books by John Arundel

You can find more Go tutorials and exercises here:

- Go tutorials from Bitfield

I have a YouTube channel where I post occasional videos on Go, and there are also some curated playlists of what I judge to be the very best Go talks and tutorials available, here:

- Bitfield Consulting on YouTube

Credits

Gopher images by the magnificent egonelbre and MariaLetta.